RWTH Aachen
Faculty of Computer Science
Chair of Combinatorial Optimization

Bachelor Thesis in Computer Science

Robust Spanning Trees

under Discrete and Interval Uncertainty
Min-Max and Min-Max Regret Models

Submitted by
Archit Dhama
Matriculation Number: 396580
archit.dhama@rwth-aachen.de

First Examiner
Prof. Dr. Christina Büsing
Chair of Combinatorial Optimization
RWTH Aachen

Second Examiner
Prof. Dr. Arie M. C. A. Koster
Chair of Discrete Optimization
RWTH Aachen

Aachen, 19th August 2026

Abstract

The minimum spanning tree (MST) connects all network nodes at minimum cost, but edge costs are often uncertain due to market fluctuations, estimation errors, or operational changes. This thesis studies robust spanning trees primarily under two uncertainty models (discrete scenarios and interval ranges) and two objectives (min-max, min-max regret), classifying computational complexity and approximability across this design space; a third model, budgeted uncertainty, receives a focused complementary treatment.

We provide a self-contained treatment from first principles. After establishing MST foundations with five complete proofs (fundamental cycle and cut lemmas, three optimality criteria via exchange arguments), we systematically analyse the resulting problem variants. For intervals, we prove extremal lemmas showing worst cases occur at boundaries: min-max becomes polynomial while regret remains NP-hard with a 2-approximation. For discrete scenarios, we prove weak NP-hardness for $K = 2$ via partition reduction and survey results for larger scenario counts, from pseudo-polynomial algorithms and approximation schemes at constant K to strong NP-hardness and a logarithmic approximation guarantee once K is part of the input. The budgeted case is shown to inherit polynomial-time solvability for min-max through a citation-led reduction to a family of deterministic minimum spanning tree computations.

The results carrying the development are proved in full: the five MST foundations, the interval extremal characterisations, two representative hardness reductions (min-max and min-max regret at $K = 2$), and the 2-approximation with its two supporting lemmas; the remaining complexity and approximability results are curated from the literature and cited. A comprehensive classification table synthesises the complexity landscape. A fixed four-vertex micro-graph illustrates concepts throughout. This work balances mathematical rigour with pedagogical clarity, serving as a focused entry point into robust combinatorial optimisation.

Contents

Abstract **i**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Research Questions and Scope . . . 2
1.3 Contributions . . . 3
1.4 Thesis Structure . . . 4

2 Foundations **5**
2.1 Graphs, Trees, and Notation . . . 5
2.2 MST Optimality Criteria . . . 8
2.3 Kruskal and Prim: Algorithmic Remarks . . . 12
2.4 Uncertainty Models and Robust Optimisation . . . 13
2.5 Algorithmic Preliminaries . . . 16
Summary . . . 18

3 Min-Max Spanning Tree **19**
3.1 Problem Formulation . . . 19
3.2 Interval Worst-Case Characterisation . . . 21
3.3 Complexity and Approximation . . . 22
3.3.1 Discrete Scenarios: Constant K . . . 23
3.3.2 Discrete Scenarios: Unbounded K . . . 26
3.4 Budgeted Uncertainty . . . 29
Summary . . . 35

4 Min-Max Regret Spanning Tree **37**
4.1 Regret Formulation . . . 37
4.2 Interval Regret Extremal Behaviour . . . 37
4.3 Complexity for Discrete Scenarios . . . 37
4.3.1 Constant K . . . 37
4.3.2 Unbounded K . . . 37
4.4 Interval Regret Approximation . . . 37
Summary . . . 37

5 Conclusion **38**
5.1 Synthesis of Results . . . 38
5.2 Example Gallery . . . 38
5.3 Limitations . . . 38
5.4 Outlook . . . 38

A Notation **40**

Declaration on the Use of AI Tools **44**

Chapter 1

Introduction

1.1 Motivation

Consider the challenge of designing a fibre-optic telecommunications network connecting major cities across a region. The network must link all locations while minimising total cable installation cost. However, actual deployment costs depend on numerous factors: terrain characteristics (installing cables through mountainous regions costs substantially more than across plains), regulatory approvals that vary by jurisdiction, and material prices that fluctuate with global supply chains. Such cost uncertainty is pervasive in infrastructure planning; as Kouvelis and Yu [1] observe, forecasted parameter values rarely match realised costs in practice, motivating robust decision-making approaches. Consequently, a network design that appears optimal under today's cost estimates may become substantially more expensive if realised costs differ significantly. This uncertainty raises a fundamental question: how should one design infrastructure networks that remain near-optimal despite cost variations?

When edge costs are known precisely, the *minimum spanning tree* (MST) provides the classical solution. The MST connects all nodes at minimum total cost and can be computed efficiently using well-established algorithms such as Kruskal's greedy edge selection or Prim's incremental tree growing, both running in polynomial time. However, in practice, costs are rarely known with certainty at design time. A tree that is optimal for one cost estimate may perform poorly when costs deviate, potentially incurring substantially higher expenses or missing better alternatives. Thus, while the MST problem is well understood in the deterministic setting, real-world applications demand solutions that are *robust* to cost uncertainty.

Under uncertainty, two natural design objectives emerge. The first is the *min-max* objective, which seeks to minimise the worst-case absolute cost across all possible cost realisations. This conservative approach hedges against expensive scenarios by selecting a spanning tree whose cost remains controlled even in the most unfavourable circumstances. Mathematically, given a set of possible cost vectors, the goal is to find a tree that minimises the maximum cost it could incur.

The second objective is *min-max regret*, which measures performance not in absolute terms but relative to what could have been achieved with perfect hindsight. Regret quantifies the difference between a chosen solution's cost and the optimal cost for a given realisation. The min-max regret objective minimises the worst-case performance gap, hedging against the possibility of missing the truly optimal solution.

Unlike min-max, which guards against high absolute costs, regret focuses on relative performance: ensuring that no matter which cost scenario materialises, the chosen tree is competitive with the scenario-optimal solution.

To model uncertainty, we consider two standard representations that arise naturally in applications, complemented later by a third that interpolates between them. The first is *discrete scenario uncertainty*, where a planner enumerates a finite set of plausible cost vectors, each representing a different potential future state. For instance, in the telecommunications example, scenarios might correspond to different combinations of regulatory outcomes and material price levels. The second is *interval uncertainty*, where each edge cost is known to lie within a specified range, bounded by lower and upper estimates. This representation is particularly natural when costs are subject to continuous variation within known limits, such as fluctuations in commodity prices or exchange rates.

Together, these two objectives (min-max and min-max regret) and the uncertainty models studied in this thesis (discrete scenarios, interval bounds, and budgeted uncertainty) define a design space of robust spanning tree variants. The central question addressed in this thesis is: how does computational complexity and approximability vary across this space? Which combinations admit polynomial-time algorithms, and which are computationally intractable? For the hard cases, what approximation guarantees can be achieved? These questions form the core of robust spanning tree optimisation and motivate the systematic classification developed in the following chapters.

1.2 Research Questions and Scope

This thesis addresses three core questions that arise naturally from the robust spanning tree framework outlined above.

First, what is the computational complexity of finding optimal robust spanning trees under the uncertainty models studied here? For each combination of objective (min-max or regret) and uncertainty model (discrete, interval, or budgeted), does there exist a polynomial-time algorithm, or is the problem computationally intractable? When hardness arises, does it stem from the number of scenarios being unbounded, or is even the case of two scenarios already difficult? Answering these questions requires establishing precise complexity classifications: polynomial-time solvability, weak NP-hardness (admitting pseudo-polynomial algorithms), or strong NP-hardness (remaining hard even with unary-encoded inputs).

Second, where do worst-case costs occur within interval uncertainty? When each edge cost lies in a specified range, must one consider all points within the Cartesian product of intervals, or can worst cases be characterised more simply? This question is critical because if worst cases always occur at interval boundaries, such as the upper or lower bounds, then the infinite continuous uncertainty set can be reduced to a finite discrete problem. Extremal lemmas that identify such worst-case locations are therefore essential tools for both algorithmic design and theoretical analysis.

Third, for problems that are computationally hard, what approximation guarantees can be achieved? Can we design polynomial-time algorithms that provably compute solutions within a constant factor of optimal, or does the problem admit a fully polynomial-time approximation scheme (FPTAS) that achieves arbitrarily

good approximations in time polynomial in both problem size and accuracy? Conversely, are there hardness-of-approximation results that limit what can be achieved? These questions determine the boundary between tractable and intractable robust optimisation.

The scope of this thesis is deliberately focused. We restrict attention to spanning tree problems in undirected, connected graphs with static, single-stage decisions: a tree is selected once, before any uncertainty is resolved, and remains fixed. The uncertainty models studied are discrete scenarios (a finite collection of cost vectors), interval bounds (per-edge lower and upper limits forming a Cartesian product), and budgeted uncertainty (intervals with an additional cap on the total deviation, introduced briefly in Section 3.4). The objectives analysed are min-max (minimising worst-case absolute cost) and min-max regret (minimising worst-case performance gap against scenario-optimal solutions).

This thesis does not cover polyhedral uncertainty sets, ellipsoidal uncertainty, or distributional robustness (where costs follow known probability distributions). It also excludes two-stage and recoverable optimisation models, where decisions can be partially adjusted after observing realisations. These extensions represent active research directions and are briefly discussed in the outlook (Section 5.4), but they lie outside the core scope of this work. The goal is not encyclopaedic coverage but rather a self-contained, rigorous treatment of the selected models, proving foundational results from first principles and synthesising established complexity and approximability findings into a unified classification.

1.3 Contributions

This thesis delivers five principal contributions to the literature on robust spanning tree optimisation.

First, we provide self-contained minimum spanning tree foundations (Chapter 2). We prove five fundamental results from first principles: the fundamental cycle and cut lemmas via exchange arguments, and three MST optimality criteria (cycle property, cut property, and their equivalence). Chapter 2 also includes a complexity primer defining polynomial time, NP-hardness (weak and strong), and approximation concepts (constant-factor algorithms, FPTAS, PTAS) for classifying the robust variants.

Second, we establish extremal characterisations for interval uncertainty (Chapters 3 and 4). For the min-max objective, Lemma 3.1 proves that worst-case costs occur when chosen edges are assigned their upper bounds, immediately yielding a polynomial-time algorithm. For the regret objective, Lemma 4.1 proves that worst-case regret occurs at boundary vertices of the interval box, though determining which boundary is harder. Both lemmas are proved in full, and their implications for algorithm design are developed carefully.

Third, we provide two representative complexity proofs (Chapters 3 and 4). Theorem 3.1 establishes weak NP-hardness of min-max spanning trees with two discrete scenarios via a reduction from the partition problem, constructing a grid graph where scenario costs encode target subset sums. The proof includes the reduction construction, correctness argument, and encoding analysis. Theorem 4.1 extends this to min-max regret by reusing the construction with adjusted analysis,

demonstrating that regret complexity mirrors absolute cost complexity for discrete scenarios. Together with cited results for larger scenario counts, these establish a complete complexity hierarchy.

Fourth, we prove a 2-approximation algorithm for interval regret spanning trees (Chapter 4). Theorem 4.5 demonstrates that solving the MST problem at midpoint costs yields a solution whose worst-case regret is at most twice optimal. The proof is developed rigorously through two supporting lemmas (Lemmas 4.2 and 4.3) bounding the midpoint solution's regret from below and above. This factor of 2 is the best approximation guarantee known for the interval regret spanning tree problem, and whether it can be improved remains an open question in the literature.

Fifth, we synthesise results into a comprehensive classification table (Chapter 5). Table 5.1 organises problem variants by objective and uncertainty model with their complexity classes and approximation guarantees. The table reveals structural patterns: intervals and budgeted uncertainty exhibit extremal behaviour enabling tractability for min-max, scenario count K acts as a complexity parameter (constant versus unbounded) for discrete uncertainty, and the two objectives display parallel hardness for discrete scenarios but diverge for intervals.

Throughout the thesis, a fixed four-vertex micro-graph with interval costs, derived discrete scenarios, and derived budgeted parameters (nominal costs and maximum deviations) provides worked examples illustrating optimal solutions under each objective and uncertainty model. This pedagogical device grounds abstract theoretical results in concrete calculations, demonstrating that robust optima genuinely differ from nominal solutions.

1.4 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 establishes mathematical foundations, proving minimum spanning tree optimality criteria from first principles and defining the robust optimisation framework (uncertainty models, objectives, complexity terminology).

Chapter 3 analyses the min-max objective, characterising worst-case costs for interval uncertainty and establishing computational complexity for discrete scenarios across different values of the scenario count parameter.

Chapter 4 examines the min-max regret objective, which measures relative performance rather than absolute cost. Interval regret is shown to be harder than interval min-max despite sharing extremal properties, while discrete regret mirrors the complexity hierarchy of discrete min-max.

Chapter 5 consolidates the results from Chapters 3 and 4 into a comprehensive classification table, displays the running micro-graph under each robust objective in an example gallery, acknowledges the deliberate scope limitations of this work, and points to related robust optimisation models (polyhedral uncertainty, two-stage formulations, recoverable robustness) along with open questions from the literature.

Chapter A collects the notation of the thesis in a single table, grouped by category, with the section of first use given for each symbol.

Chapter 2

Foundations

This chapter develops the foundations needed for analysing robust spanning tree problems. After fixing graph-theoretic notation and introducing the running micrograph (Section 2.1), we prove the MST optimality criteria from first principles (Section 2.2) and remark on the classical greedy algorithms they justify (Section 2.3). Section 2.4 then formalises the three uncertainty models, discrete scenarios, interval uncertainty, and budgeted uncertainty, together with the min-max and min-max regret objectives. Section 2.5 reviews the complexity and approximation terminology used in Chapters 3 and 4.

2.1 Graphs, Trees, and Notation

We work exclusively with undirected graphs throughout this thesis. A *graph* is a pair $G = (V, E)$, where V is a finite non-empty set of *vertices* (or *nodes*) and $E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ is a set of *edges*. Each edge connects exactly two distinct vertices; we write $\{u, v\} \in E$ or equivalently $uv \in E$ to denote an edge between u and v. A graph is called *simple* if it contains no multiple edges (at most one edge between any pair of vertices) and no loops (edges connecting a vertex to itself). All graphs considered in this thesis are simple. We denote $n = |V|$ and $m = |E|$ for the number of vertices and edges, respectively. Two vertices $u, v \in V$ are *adjacent* (or *neighbours*) if $uv \in E$, and the *degree* of a vertex v, written $\deg(v)$, is the number of edges incident to v.

A *path* in G is a sequence of distinct vertices $v_0, v_1, \ldots, v_k$ such that $\{v_{i-1}, v_i\} \in E$ for all $i \in \{1, \ldots, k\}$. The *length* of this path is k (the number of edges). A *cycle* is a sequence of distinct vertices $v_0, v_1, \ldots, v_k$ with $k \geq 2$ such that $\{v_{i-1}, v_i\} \in E$ for all $i \in \{1, \ldots, k\}$ and $\{v_k, v_0\} \in E$.

A graph G is *connected* if for every pair of vertices $u, v \in V$, there exists a path from u to v. A spanning tree can only exist if G is connected, since the tree must reach every vertex. **Throughout this thesis, we assume all graphs are connected.**

For a non-empty proper subset $X \subseteq V$ (that is, $\emptyset \neq X \subsetneq V$), the *cut* induced by X is the set of edges crossing the partition $(X, V \setminus X)$:

$$\delta(X) = \{\{u, v\} \in E \mid u \in X,\, v \in V \setminus X\}.$$

Intuitively, $\delta(X)$ comprises all edges with exactly one endpoint in X. See Figure 2.1 for a visual representation.

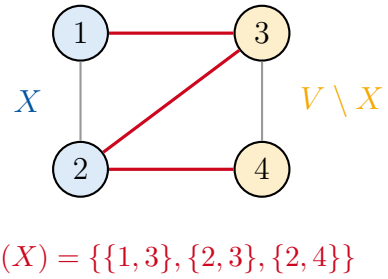

Figure 2.1: Illustration of a cut $\delta(X)$ induced by vertex set $X = \{1, 2\}$. The cut $\delta(X)$ consists of the three red edges connecting the blue-shaded vertices in X to the orange-shaded vertices in $V \setminus X$.

Each edge $e \in E$ has an associated *cost* $c_e \in \mathbb{R}$. We represent the cost structure either as a *cost function* $c\colon E \to \mathbb{R}$ or equivalently as a *cost vector* $c \in \mathbb{R}^{|E|}$ indexed by the edges. Costs may be positive, negative, or zero in the deterministic setting; the robust models in Section 2.4 will additionally impose non-negative costs. For any subset $F \subseteq E$, we write $c(F) = \sum_{e \in F} c_e$ for the total cost of the edge set F.

Spanning Trees. A *spanning tree* of G is a connected acyclic subgraph $T \subseteq G$ that includes all vertices of G, that is, $V(T) = V(G)$. We denote by $\mathcal{T}$ the set of all spanning trees of G. Since spanning trees are both connected and acyclic, they satisfy precisely $|E(T)| = n - 1$ edges, where $E(T) \subseteq E$ denotes the edge set of T. These three properties, connectedness, acyclicity, and having exactly $n - 1$ edges, are not independent: for a subgraph spanning all n vertices, any two of them imply the third [2, Theorem 2.1]. We use this fact repeatedly when verifying that a constructed subgraph is a spanning tree. For a spanning tree $T \in \mathcal{T}$, we write $c(T) = c(E(T)) = \sum_{e \in E(T)} c_e$ for the cost of T.

The *minimum spanning tree problem* (MST) seeks a spanning tree of minimum cost:

$$\mathrm{MST}(c) = \min_{T \in \mathcal{T}} c(T). \tag{2.1}$$

Any tree $T^* \in \mathcal{T}$ satisfying $c(T^*) = \mathrm{MST}(c)$ is called an *MST* under cost vector c. The MST is unique if all edge costs are distinct; when edge costs have ties (equal values), multiple spanning trees may achieve the minimum cost $\mathrm{MST}(c)$.

A spanning tree T possesses three fundamental structural properties that we exploit repeatedly:

(P1) For any two vertices $u, v \in V$, there exists a *unique* path in T connecting u and v.

(P2) Removing any edge $e \in E(T)$ disconnects T into exactly two connected components.

(P3) Adding any edge $f \in E \setminus E(T)$ to T creates exactly one cycle.

Property P1 follows from the definition of trees as connected acyclic graphs: connectivity ensures path existence, while acyclicity guarantees uniqueness (a second path would form a cycle). Properties P2 and P3 are immediate consequences: removing an edge from a minimally connected subgraph (a tree on n vertices has exactly $n-1$ edges) must disconnect it, while adding an edge to a maximally acyclic subgraph must create a cycle [2, Theorem 2.1].

Fundamental Structures. We now introduce two graph-theoretic constructs, fundamental cycles and fundamental cuts, that play a central role in characterising MST optimality. Let $T \in \mathcal{T}$ be a spanning tree and let $f \in E \setminus E(T)$ be an edge not in T with endpoints u and v. By property P3, adding f to T creates a unique cycle. Since T is a tree, by property P1, there exists a unique path P in T from u to v. We call the cycle $C_f = E(P) \cup \{f\}$ (where $E(P)$ denotes the edges of path P) the *fundamental cycle* of f with respect to T.

Similarly, let $e \in E(T)$ be an edge in the tree T. By property P2, removing e from T partitions V into exactly two connected components. Let $X_e \subseteq V$ denote one of these components (the choice is arbitrary; either component induces the same cut). The *fundamental cut* of e with respect to T is the cut $\delta(X_e)$ induced by X_e. Crucially, e is the *only* tree edge in $\delta(X_e)$; all other edges in the cut belong to $E \setminus E(T)$. This follows because X_e is a connected component of $T \setminus \{e\}$: any other tree edge crossing the partition would connect X_e to $V \setminus X_e$, contradicting the component structure. These fundamental structures underpin the MST optimality criteria developed in Section 2.2: fundamental cycles correspond to potential edge exchanges for non-tree edges, while fundamental cuts correspond to exchanges for tree edges.

Running Example: The Micro-graph. To anchor the concepts of this thesis, we fix a four-vertex graph that recurs throughout as a running example. Let $G = (V, E)$ with vertex set $V = \{1, 2, 3, 4\}$ and five edges:

$$e_1 = \{1, 2\}, \quad e_2 = \{2, 3\}, \quad e_3 = \{2, 4\},$$
$$e_4 = \{3, 4\}, \quad e_5 = \{1, 3\}.$$

The structure is depicted in Figure 2.2. Vertex 2 serves as a central hub with edges to vertices 1, 3, and 4, while edges $e_5 = \{1, 3\}$ and $e_4 = \{3, 4\}$ provide alternative connections.

In anticipation of the robust optimisation framework of Section 2.4, we associate each edge e_i with an *interval cost* $[\ell_{e_i}, u_{e_i}]$ representing uncertainty in edge costs:

$$e_1 \colon [2, 4], \quad e_2 \colon [3, 5], \quad e_3 \colon [1, 7],$$
$$e_4 \colon [4, 6], \quad e_5 \colon [5, 7].$$

From these intervals, we derive three *discrete scenarios* that instantiate specific cost realisations: $c^{(1)} = (2, 3, 1, 4, 5)$ (all lower bounds), $c^{(2)} = (4, 5, 7, 6, 7)$ (all upper bounds), and $c^{(3)} = (3, 4, 4, 5, 6)$ (interval midpoints). Graph G admits eight spanning trees in total. We highlight three representative trees that exhibit different structural properties: $T_1 = \{e_1, e_2, e_3\}$ (star centred at vertex 2), $T_2 = \{e_1, e_2, e_4\}$ (path 1–2–3–4), and $T_3 = \{e_2, e_3, e_5\}$ (path 1–3–2–4). These trees and scenarios

provide a consistent testbed for demonstrating how optimal solutions vary across objectives and uncertainty models in subsequent chapters.

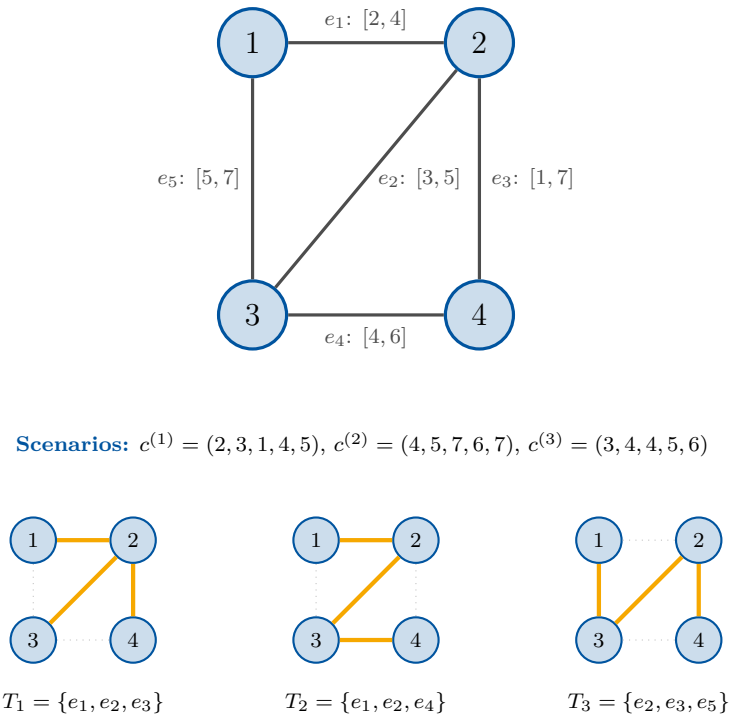

Figure 2.2: The micro-graph G with interval costs $[\ell_e, u_e]$ on each edge (top), used throughout this thesis, together with the three representative spanning trees T_1, T_2, T_3 (bottom, highlighted edges).

2.2 MST Optimality Criteria

A remarkable feature of minimum spanning trees is that optimality can be verified locally: rather than comparing a candidate tree against all exponentially many spanning trees, we can check simple conditions on individual edges. This section develops these local characterisations through five results. We begin with two fundamental exchange lemmas (Lemmas 2.1 and 2.2) that establish necessary conditions for optimality, then prove three equivalent optimality criteria (Theorems 2.1 to 2.3) that provide both necessary and sufficient conditions, thereby fully characterising MST optimality. These characterisations underpin the correctness of classical MST algorithms such as Kruskal's and Prim's greedy methods.

Our first lemma relates non-tree edges to the cycles they create.

Lemma 2.1 (Fundamental Cycle Lemma)**.** *Let $G = (V, E)$ be a connected graph with cost function $c\colon E \to \mathbb{R}$, and let T be a minimum spanning tree of G under c. For any edge $f \in E \setminus E(T)$, the edge f has maximum cost among all edges in its fundamental cycle C_f. That is, $c_f \geq c_e$ for every edge $e \in C_f$.*

Proof. Suppose for contradiction that f does not have maximum cost in C_f. Then there exists an edge $e \in C_f$ with $c_e > c_f$. Since $f \notin E(T)$ and C_f is created by adding f to T, the edge e must be a tree edge.

We construct an alternative spanning tree by exchanging e for f. Removing e from T disconnects T into two components; since e lies on the path connecting the endpoints of f in T, the edge f bridges these components. Define T' with edge set $E(T') = E(T) \setminus \{e\} \cup \{f\}$.

To verify that T' is a spanning tree, it suffices to check two of the three defining properties. First, $|E(T')| = |E(T)| - 1 + 1 = n - 1$, as required for a tree on n vertices. Second, T' is connected: removing e partitions V into two components, but f crosses this partition (as f completes the cycle C_f containing e), so adding f reconnects the graph. Having $n-1$ edges and being connected, T' is a spanning tree.

The cost of T' is

$$c(T') = c(T) - c_e + c_f < c(T),$$

where the inequality holds because $c_e > c_f$. This contradicts the optimality of T. Therefore, f must have maximum cost in C_f. □

An analogous statement holds for tree edges, with the fundamental cut $\delta(X_e)$ (introduced in Section 2.1) playing the role that the fundamental cycle played for non-tree edges.

Lemma 2.2 (Fundamental Cut Lemma)**.** *Let $G = (V, E)$ be a connected graph with cost function $c\colon E \to \mathbb{R}$, and let T be a minimum spanning tree of G under c. For any edge $e \in E(T)$, the edge e has minimum cost among all edges in its fundamental cut $\delta(X_e)$. That is, $c_e \leq c_f$ for every edge $f \in \delta(X_e)$.*

Proof. Suppose for contradiction that e does not have minimum cost in $\delta(X_e)$. Then there exists an edge $f \in \delta(X_e)$ with $c_f < c_e$. By construction of the fundamental cut (Section 2.1), edge e is the unique tree edge in $\delta(X_e)$, so f must be a non-tree edge.

We construct an alternative spanning tree by exchanging e for f. Removing e from T splits V into components X_e and $V \setminus X_e$, which f bridges (since $f \in \delta(X_e)$). Define T' with edge set $E(T') = E(T) \setminus \{e\} \cup \{f\}$.

To verify that T' is a spanning tree, it suffices to check two of the three defining properties. First, $|E(T')| = |E(T)| - 1 + 1 = n - 1$, as required for a tree on n vertices. Second, T' is connected: f has one endpoint in each component (by definition of $\delta(X_e)$), so adding f restores connectivity. Having $n-1$ edges and being connected, T' is a spanning tree.

The cost of T' is

$$c(T') = c(T) - c_e + c_f < c(T),$$

where the inequality holds because $c_f < c_e$. This contradicts the optimality of T. Therefore, e must have minimum cost in $\delta(X_e)$. □

These fundamental exchange properties establish necessary conditions for a tree to be an MST. The optimality criteria developed next strengthen these to necessary and sufficient characterisations.

Remark 2.1 (Cost Ties). The fundamental lemmas use non-strict inequalities ($c_f \geq c_e$ in Lemma 2.1; $c_e \leq c_f$ in Lemma 2.2), since multiple edges may share the same cost. When all edge costs are distinct, the MST is unique and the inequalities can be tightened to strict; when ties exist, multiple spanning trees may achieve the minimum cost, and the lemmas apply to each of them.

Theorem 2.1 (Cycle Criterion)**.** *Let $G = (V, E)$ be a connected graph with cost function $c\colon E \to \mathbb{R}$, and let T be a spanning tree of G. Then T is a minimum spanning tree if and only if for every non-tree edge $f \in E \setminus E(T)$, the edge f has maximum cost among all edges in its fundamental cycle C_f.*

Proof. We prove both directions.

($\Rightarrow$)**:** If T is a minimum spanning tree, then every non-tree edge has maximum cost in its fundamental cycle by Lemma 2.1.

($\Leftarrow$)**:** Suppose every non-tree edge has maximum cost in its fundamental cycle. We show T is a minimum spanning tree by proving $c(T) \leq c(T')$ for any spanning tree T'.

If $T = T'$, we are done. Otherwise, there exists an edge $e \in E(T) \setminus E(T')$. Write $e = \{s, t\}$ and let $X_e \subseteq V$ denote the component containing s after removing e from T, so that $t \in V \setminus X_e$. Since T' is connected, there exists a path P in T' from s to t. Because s and t lie on opposite sides of the partition $(X_e, V \setminus X_e)$, the path P must cross it at least once.

Pick any edge f on P that crosses the partition. Then f has one endpoint in X_e and the other in $V \setminus X_e$, hence $f \in \delta(X_e)$. Moreover, $f \in E(T')$ (since f lies on the path P) and $f \notin E(T)$ (since e is the unique tree edge in $\delta(X_e)$ by construction of the fundamental cut; see Section 2.1).

We construct a new spanning tree $T'' = (V, E(T') \setminus \{f\} \cup \{e\})$ by removing f and adding e. To verify that T'' is a spanning tree, note that f lies on the path P from s to t in T', so removing f disconnects s from t, while adding $e = \{s, t\}$ reconnects them; thus T'' is connected. Since $|E(T'')| = |E(T')| - 1 + 1 = n - 1$, being connected with $n - 1$ edges makes T'' a spanning tree.

We now verify that $c(T'') \leq c(T')$. Since $f \notin E(T)$, the edge f creates a fundamental cycle C_f in T. The path in T connecting the endpoints of f must cross the partition $(X_e, V \setminus X_e)$, and because e is the unique tree edge in $\delta(X_e)$, this path contains e. Therefore $e \in C_f$. By assumption, f has maximum cost in C_f, so $c_f \geq c_e$. Hence

$$c(T'') = c(T') - c_f + c_e \leq c(T').$$

Moreover, $|E(T) \cap E(T'')| = |E(T) \cap E(T')| + 1$, since we added $e \in E(T)$ and removed $f \notin E(T)$.

Repeating this exchange, we obtain a sequence of spanning trees $T' = T_0, T_1, T_2, \ldots$ where each T_{i+1} is derived from T_i by exchanging an edge of $E(T) \setminus E(T_i)$ with an edge of $E(T_i) \setminus E(T)$, satisfying $c(T_{i+1}) \leq c(T_i)$ and $|E(T) \cap E(T_{i+1})| = |E(T) \cap E(T_i)| + 1$. Since spanning trees have exactly $n - 1$ edges, the process terminates after at most $n - 1$ iterations with some $T_k = T$. The cost satisfies

$$c(T) = c(T_k) \leq c(T_{k-1}) \leq \cdots \leq c(T_1) \leq c(T_0) = c(T').$$

Therefore $c(T) \leq c(T')$ for every spanning tree T', which means T is a minimum spanning tree. □

The cut-based criterion is analogous.

Theorem 2.2 (Cut Criterion)**.** *Let $G = (V, E)$ be a connected graph with cost function $c\colon E \to \mathbb{R}$, and let T be a spanning tree of G. Then T is a minimum*

spanning tree if and only if for every tree edge $e \in E(T)$, the edge e has minimum cost among all edges in its fundamental cut $\delta(X_e)$.

Proof. We prove both directions.

($\Rightarrow$)**:** If T is a minimum spanning tree, then every tree edge has minimum cost in its fundamental cut by Lemma 2.2.

($\Leftarrow$)**:** Suppose every tree edge has minimum cost in its fundamental cut. We show $c(T) \leq c(T')$ for any spanning tree T'.

The argument mirrors the proof of Theorem 2.1. If $T = T'$, we are done. Otherwise, pick any edge $e \in E(T) \setminus E(T')$ with $e = \{s, t\}$, and let X_e denote the component containing s in $T \setminus \{e\}$.

Since T' is connected, there exists a path P in T' from s to t. Because $s \in X_e$ and $t \in V \setminus X_e$ lie on opposite sides of the partition, the path P crosses $(X_e, V \setminus X_e)$ at least once. Let f be any edge on P crossing the partition. Then $f \in \delta(X_e)$, $f \in E(T')$, and $f \notin E(T)$ (since e is the unique tree edge in $\delta(X_e)$ and $f \neq e$). By the cut criterion, $c_e \leq c_f$.

By the same spanning tree verification as in Theorem 2.1, the exchange $T'' = (V, E(T') \setminus \{f\} \cup \{e\})$ yields a spanning tree with

$$c(T'') = c(T') - c_f + c_e \leq c(T')$$

and $|E(T) \cap E(T'')| = |E(T) \cap E(T')| + 1$.

Repeating this exchange, we obtain a sequence $T' = T_0, T_1, \ldots, T_k = T$ where each step satisfies $c(T_{i+1}) \leq c(T_i)$ and increases $|E(T) \cap E(T_i)|$ by one. The process terminates when $T_k = T$, yielding

$$c(T) = c(T_k) \leq c(T_{k-1}) \leq \cdots \leq c(T_0) = c(T').$$

Therefore $c(T) \leq c(T')$ for every spanning tree T', so T is a minimum spanning tree. □

The two criteria are equivalent.

Theorem 2.3 (Equivalence of Criteria)**.** *Let $G = (V, E)$ be a connected graph with cost function $c\colon E \to \mathbb{R}$, and let T be a spanning tree of G. Then T satisfies the cycle criterion if and only if T satisfies the cut criterion.*

Proof. By Theorem 2.1, T satisfies the cycle criterion if and only if T is a minimum spanning tree. By Theorem 2.2, T satisfies the cut criterion if and only if T is a minimum spanning tree. Therefore, T satisfies the cycle criterion if and only if T satisfies the cut criterion. □

Together, these three theorems fully characterise minimum spanning tree optimality through local edge-level conditions, enabling verification without comparing against all exponentially many spanning trees. The characterisations extend naturally to the robust spanning tree problems in Chapters 3 and 4, where similar exchange arguments establish optimality under uncertainty. The correctness of classical greedy algorithms such as Kruskal's and Prim's follows from these criteria, as discussed in Section 2.3.

2.3 Kruskal and Prim: Algorithmic Remarks

The optimality criteria developed in Section 2.2 directly yield the correctness of the two most classical MST algorithms: Kruskal's algorithm (1956) and Prim's algorithm (1957). Both are *greedy* methods that build a spanning tree incrementally, maintaining optimality conditions throughout [3, §2.3]. A comprehensive account of minimum spanning tree algorithms is given by Ahuja, Magnanti and Orlin [4, Chapter 13]. We describe each algorithm briefly, emphasising its connection to the criteria rather than implementation details.

Kruskal's Algorithm. Kruskal's algorithm constructs an MST by processing edges in non-decreasing order of cost. Starting with an empty edge set, the algorithm considers each edge in turn and adds it to the current forest if and only if it does not create a cycle. The process terminates when the forest becomes a spanning tree (after adding exactly $n - 1$ edges).

The correctness follows immediately from the cycle criterion (Theorem 2.1). When the algorithm terminates with spanning tree T, consider any non-tree edge $f \in E \setminus E(T)$. Since f was rejected, adding f would have created a cycle with edges already in T. All edges in this cycle were added before f was considered, hence each has cost at most c_f (by the sorting order). Therefore f has maximum cost in its fundamental cycle, satisfying the cycle criterion.

With an efficient union-find data structure to track connected components, Kruskal's algorithm runs in $O(m \log n)$ time, dominated by the initial edge sorting [2, Theorem 6.5].

Prim's Algorithm. Prim's algorithm grows a single tree from an arbitrary starting vertex. At each step, it adds the minimum-cost edge crossing the cut between the current tree vertices and the remaining vertices. The process continues until all vertices are included.

The correctness follows by induction on the steps of the algorithm. Let S_t denote the set of tree vertices before step t and e_t the edge added at step t. We claim that the partial tree built so far can be extended to some MST of G at every step; since the algorithm terminates with a spanning tree, this implies the final tree is itself an MST.

The base case is trivial: the empty tree on the chosen root extends to any MST. For the inductive step, assume the partial tree T_{t-1} is contained in some MST T^*, and consider the edge e_t added next. If $e_t \in E(T^*)$, the induction continues with the same T^*.

Otherwise $e_t \notin E(T^*)$. The unique path in T^* between the endpoints of e_t starts in S_{t-1} (one endpoint of e_t lies there) and ends outside S_{t-1} (the other endpoint is the newly added vertex), so it must cross the cut $\delta(S_{t-1})$ at some edge $f \in E(T^*)$. By Prim's choice, e_t is the minimum-cost edge in $\delta(S_{t-1})$, hence $c_{e_t} \leq c_f$. The exchange $T^{**} = (T^* \setminus \{f\}) \cup \{e_t\}$ is again a spanning tree: adding e_t to T^* creates exactly its fundamental cycle, and f lies on that cycle, so removing f restores acyclicity. Its cost satisfies $c(T^{**}) = c(T^*) - c_f + c_{e_t} \leq c(T^*)$, so T^{**} is also an MST and contains $T_{t-1} \cup \{e_t\}$.

By induction, the spanning tree built by Prim's algorithm is an MST.

With a priority queue maintaining the minimum-cost edge to each non-tree vertex, Prim's algorithm achieves $O(m + n^2)$ time with a simple array implementation, or $O(m + n \log n)$ with Fibonacci heaps [2, Theorem 6.7].

Relevance to Robust Problems. Both algorithms solve the deterministic MST problem optimally in polynomial time. However, under uncertainty, the situation changes fundamentally. When edge costs are uncertain, neither algorithm directly applies: the greedy choices depend on which cost realisation occurs, yet the spanning tree must be chosen before costs are revealed. The robust spanning tree problems studied in Chapters 3 and 4 address precisely this challenge, and several variants become NP-hard despite the tractability of the deterministic case.

2.4 Uncertainty Models and Robust Optimisation

In many practical applications, edge costs are not known precisely at decision time. Construction expenses may fluctuate, travel times depend on traffic conditions, and communication link costs vary with demand. Robust optimisation addresses this challenge by seeking solutions that perform well across all possible cost realisations, rather than optimising for a single nominal scenario [1]. This section formalises three uncertainty models and the corresponding robust objectives that we analyse in Chapters 3 and 4. All three require non-negative edge costs, reflecting the distances, construction expenses, and transmission delays that such costs typically represent, as is standard in the robust spanning tree literature [1, 5].

Discrete Scenarios. In the *discrete scenario model*, uncertainty is represented by a finite set of possible cost vectors.

Definition 2.1 (Discrete Uncertainty Set)**.** *A discrete uncertainty set consists of K explicitly given cost vectors:*

$$\mathcal{U} = \{c^{(1)}, c^{(2)}, \ldots, c^{(K)}\},$$

where each scenario $c^{(k)}$, for $k \in \{1, \ldots, K\}$, is a cost vector in $\mathbb{R}_{\geq 0}^{|E|}$ assigning cost $c_e^{(k)}$ to every edge $e \in E$.

The parameter K denotes the number of scenarios. When K is a fixed constant (such as $K = 2$), we speak of a *constant number of scenarios*; when K is part of the input and may grow with the problem size, we speak of *unbounded* K. This distinction is crucial for computational complexity: several robust spanning tree problems are weakly NP-hard already for a constant number of scenarios, admitting pseudo-polynomial algorithms and approximation schemes, but become strongly NP-hard when K is unbounded [6].

Interval Uncertainty. In the *interval model*, each edge cost may take any value within a specified range.

Definition 2.2 (Interval Uncertainty Set)**.** *An interval uncertainty set is defined by lower and upper bounds on each edge:*

$$\mathcal{U} = \prod_{e \in E} [\ell_e, u_e] = \left\{ c \in \mathbb{R}^{|E|}_{\geq 0} : \ell_e \leq c_e \leq u_e \text{ for all } e \in E \right\},$$

where $0 \leq \ell_e \leq u_e$ *for each edge* $e \in E$*.*

Unlike the discrete model with finitely many scenarios, interval uncertainty encompasses a continuum of possible cost vectors. Despite this, many robust problems under interval uncertainty admit efficient solutions by exploiting the structure of the uncertainty set: worst-case costs are often attained at the boundary of the intervals. We develop these arguments in Chapters 3 and 4.

Budgeted Uncertainty. In the *budgeted model*, each edge has a baseline cost together with a bound on how far it may rise, and a single parameter caps how many edges may deviate from baseline at once.

Definition 2.3 (Budgeted Uncertainty Set)**.** *Let* $\bar{c} \in \mathbb{R}^{|E|}_{\geq 0}$ *be a vector of nominal costs (the baseline value of each edge),* $\hat{c} \in \mathbb{R}^{|E|}_{\geq 0}$ *a vector of maximum deviations (the largest upward perturbation each edge can suffer), and* $\Gamma \in [0, |E|]$ *a deviation budget. The budgeted uncertainty set is*

$$\mathcal{U}^{\Gamma} = \left\{ c \in \mathbb{R}^{|E|}_{\geq 0} : c_e = \bar{c}_e + \hat{c}_e \, \delta_e, \;\; \delta_e \in [0,1] \text{ for } e \in E, \;\; \sum_{e \in E} \delta_e \leq \Gamma \right\}.$$

Each edge cost c_e is determined by a deviation factor $\delta_e \in [0,1]$: at $\delta_e = 0$ the edge takes its nominal cost $\bar{c}_e$, at $\delta_e = 1$ it takes the worst-case value $\bar{c}_e + \hat{c}_e$, and intermediate values interpolate linearly. The budget constraint $\sum_e \delta_e \leq \Gamma$ limits the cumulative deviation across the graph. Setting $\Gamma = 0$ recovers the deterministic problem on $\bar{c}$, while $\Gamma = |E|$ allows every edge to deviate independently and recovers an interval model with $[\ell_e, u_e] = [\bar{c}_e, \bar{c}_e + \hat{c}_e]$ [5]. The budget Γ thus offers a tunable middle ground: the modeller controls how conservative the analysis is through a single parameter, rather than committing in advance to a fixed scenario list or to the full interval ranges. As we shall see in Section 3.4, this added structure allows the min-max spanning tree problem to remain polynomial-time solvable under budgeted uncertainty.

Robust Objectives. Given an uncertainty set $\mathcal{U}$, we model robust optimisation as a sequential game: a decision-maker first selects a spanning tree T, then an adversary selects a worst-case scenario $c \in \mathcal{U}$. This structure motivates two robust objectives.

The *min-max objective* minimises the maximum cost over all scenarios:

$$\min_{T \in \mathcal{T}} \max_{c \in \mathcal{U}} c(T). \tag{2.2}$$

A spanning tree achieving this minimum is called a *min-max spanning tree*. This objective bounds the worst possible outcome regardless of which scenario materialises.

The *min-max regret objective* instead minimises the maximum *regret*, which measures how far a solution falls short of the scenario-specific optimum:

$$\text{Regret}(T, c) = c(T) - \text{MST}(c). \tag{2.3}$$

The min-max regret problem seeks a tree minimising the worst-case regret:

$$\min_{T \in \mathcal{T}} \max_{c \in \mathcal{U}} \text{Regret}(T, c) = \min_{T \in \mathcal{T}} \max_{c \in \mathcal{U}} \Big[c(T) - \text{MST}(c) \Big]. \tag{2.4}$$

A spanning tree achieving this minimum is called a *min-max regret spanning tree*. This objective minimises *opportunity cost*: the loss from not knowing the scenario in advance.

Computing regret requires knowing $\text{MST}(c)$, which depends on the scenario c. For discrete scenarios, we can precompute the MST for each scenario separately. For interval and budgeted uncertainty, the adversary's choice of c affects both $c(T)$ and $\text{MST}(c)$ simultaneously, and the worst-case scenario is generally not simply all upper bounds; identifying it requires the extremal arguments developed in Chapters 3 and 4.

Worked Example: Discrete Scenarios on the Micro-graph. We illustrate the robust objectives using the discrete scenario model on the micro-graph from Section 2.1. (Worked examples for interval and budgeted uncertainty require the extremal analysis developed in Chapters 3 and 4; the budgeted case is treated in Section 3.4.)

The micro-graph has vertices $V = \{1, 2, 3, 4\}$, edges $\{e_1, \ldots, e_5\}$, and interval costs as specified in Figure 2.2. We derive three discrete scenarios: $c^{(1)} = (2, 3, 1, 4, 5)$ (all lower bounds), $c^{(2)} = (4, 5, 7, 6, 7)$ (all upper bounds), and $c^{(3)} = (3, 4, 4, 5, 6)$ (interval midpoints). We evaluate three spanning trees: $T_1 = \{e_1, e_2, e_3\}$, $T_2 = \{e_1, e_2, e_4\}$, and $T_3 = \{e_2, e_3, e_5\}$.

Table 2.1 records the cost and regret for each tree–scenario pair. The bottom row shows the MST cost for each scenario; note that the MST varies: T_1 is optimal under $c^{(1)}$ and $c^{(3)}$, whereas under $c^{(2)}$ the tree T_2 achieves the minimum cost 15.

Table 2.1: Costs and regrets for representative spanning trees under three discrete scenarios.

	Cost $c^{(k)}(T)$			Regret			Max
Tree	k=1	k=2	k=3	k=1	k=2	k=3	Regret
$T_1 = \{e_1, e_2, e_3\}$	6	16	11	0	1	0	1
$T_2 = \{e_1, e_2, e_4\}$	9	15	12	3	0	1	3
$T_3 = \{e_2, e_3, e_5\}$	9	19	14	3	4	3	4
$\text{MST}(c^{(k)})$	6	15	11				

To find the min-max regret tree, we proceed in two steps: compute each tree's maximum regret across scenarios (rightmost column), then select the tree with the

smallest maximum. Tree T_1 has max regret 1, compared to 3 for T_2 and 4 for T_3; no other spanning tree of the micro-graph does better, so T_1 is the min-max regret spanning tree.

For the min-max objective, we compare worst-case costs: T_2 achieves 15 (under $c^{(2)}$), versus 16 for T_1 and 19 for T_3. Again no other spanning tree improves on this, so T_2 is the min-max spanning tree. Here the two objectives select *different* trees: T_2 minimises worst-case cost, while T_1 minimises worst-case regret. This divergence illustrates that the choice of robust objective genuinely matters, a theme developed throughout Chapters 3 and 4.

2.5 Algorithmic Preliminaries

This section introduces the complexity and approximation terminology needed to interpret the results in Chapters 3 and 4. We focus on concepts directly relevant to robust spanning tree problems; for comprehensive treatments, see [5, Chapter 2] or [2, Chapter 15].

Polynomial Time and the Class P. An algorithm runs in *polynomial time* if, given an input of size n, it terminates in at most $O(n^k)$ steps for some fixed constant k. The class P consists of all problems solvable by a polynomial-time algorithm. Problems in P are considered tractable: their running time grows manageably with the input size. The minimum spanning tree problem lies in P, since Kruskal's algorithm solves it in $O(m \log n)$ time (Section 2.3). Introducing uncertainty, however, can push problems outside P, as we shall see in Chapters 3 and 4.

Hard Problems: NP and NP-hardness. The class NP (nondeterministic polynomial time) contains problems whose candidate solutions can be *verified* in polynomial time, even if finding such a solution may itself be hard. To make this concrete, consider the decision version of MST: given a graph $G = (V, E)$, costs c, and a budget B, does there exist a spanning tree of cost at most B? A candidate solution is a set $T \subseteq E$ of $n - 1$ edges. Verifying it requires two checks: that T is a spanning tree (one breadth-first search on the $n - 1$ edges of T runs in $O(n)$ time and confirms connectedness), and that the sum of its edge costs does not exceed B (another $O(n)$ additions). Since both checks are polynomial, the decision MST lies in NP.

A problem is NP-*hard* if every problem in NP can be *reduced* to it: transformed in polynomial time so that solving the target problem also solves the original. Informally, NP-hard problems are at least as hard as the hardest problems in NP. No polynomial-time algorithm is known for any NP-hard problem, and the widely believed conjecture $\mathsf{P} \neq \mathsf{NP}$ implies that none exists.

Weak and Strong NP-hardness. The knapsack problem is NP-hard, yet a textbook dynamic-programming algorithm solves it in $O(nW)$ time, where W is the knapsack capacity. At first sight this seems to contradict $\mathsf{P} \neq \mathsf{NP}$. The resolution lies in how input size is measured: a number W is encoded in binary using only about $\log_2 W$ bits, not W itself. Doubling W therefore adds just one bit to the input but doubles the runtime, so $O(nW)$ is polynomial in the *value* of W but exponential in the *bit-length* of W, which is what "polynomial-time" strictly demands.

An algorithm is *pseudo-polynomial* if its running time is polynomial in the input size and in the magnitudes of the numeric values appearing in the input. Such an algorithm is not polynomial in the strict sense, but it is efficient whenever those values are not too large. This motivates two refinements of NP-hardness:

- A problem is *weakly NP-hard* if it is NP-hard and admits a pseudo-polynomial algorithm. The knapsack problem is the canonical example.
- A problem is *strongly NP-hard* if it is NP-hard even when all numeric values are bounded by a polynomial in n. Such problems admit no pseudo-polynomial algorithm unless $\mathsf{P} = \mathsf{NP}$.

This distinction is significant for robust spanning trees: the number of scenarios K plays a role analogous to a numeric input value, and several variants will turn out to be weakly NP-hard (Chapters 3 and 4).

Approximation Algorithms. When an exact polynomial-time algorithm is unlikely to exist, we settle for efficient algorithms returning provably near-optimal solutions.

An *α-approximation algorithm* for a minimisation problem is a polynomial-time algorithm that, on every instance, returns a solution of cost at most α times the optimum, where $\alpha \geq 1$ is the *approximation ratio*. The ratio is fixed for the algorithm: a 2-approximation guarantees a solution at most twice as costly as the optimum, with no tighter promise. Christofides' algorithm for the metric travelling salesman problem, for instance, is a $\frac{3}{2}$-approximation [2, Theorem 21.5].

For some problems, the ratio can instead be tuned by the user. A *polynomial-time approximation scheme* (PTAS) takes any $\varepsilon > 0$ as input and returns a $(1+\varepsilon)$-approximation in time polynomial in n for each fixed ε. Setting $\alpha := 1 + \varepsilon$, this is precisely an α-approximation for any $\alpha > 1$ chosen by the user, with smaller ε yielding tighter guarantees. The catch is that the runtime may grow rapidly in $1/\varepsilon$ (for instance, $O(n^{1/\varepsilon})$), so a PTAS need not be practical for very small ε.

A *fully polynomial-time approximation scheme* (FPTAS) is a PTAS whose running time is additionally polynomial in $1/\varepsilon$. The trade-off then becomes graceful: halving ε at most polynomially increases the runtime. The knapsack problem admits an FPTAS [2, Theorem 17.9], illustrating that even some NP-hard problems can be approximated to within any desired precision in practical time. However, strongly NP-hard problems cannot have an FPTAS unless $\mathsf{P} = \mathsf{NP}$.

Relevance to This Thesis. The complexity of robust spanning tree problems hinges on two factors:

1. **Number of scenarios** K: For discrete uncertainty, several problems are already weakly NP-hard for a constant number of scenarios, admitting pseudo-polynomial algorithms and approximation schemes, and become strongly NP-hard when K is part of the input.
2. **Uncertainty model**: For interval uncertainty, the continuum of scenarios prevents enumeration. Some variants remain in P by exploiting MST structure (Section 2.2); others are NP-hard.

Chapters 3 and 4 classify each problem variant accordingly and present polynomial-time algorithms or approximations as appropriate.

Summary

Two technical pillars established in this chapter recur throughout the rest of the thesis. The first is the local characterisation of MST optimality through fundamental cycles and cuts (Theorems 2.1 and 2.2): the exchange arguments built on these criteria will reappear repeatedly in the robust setting. The second is the family of uncertainty models (discrete scenarios, interval uncertainty, and budgeted uncertainty) combined with robust objectives (min-max cost and min-max regret), which yields the problem variants analysed in Chapters 3 and 4. Several of these variants will turn out to be NP-hard, in sharp contrast to the polynomial-time tractability of the deterministic MST.

Chapter 3

Min-Max Spanning Tree

Chapter 2 established that the deterministic minimum spanning tree problem, in which all edge costs are known in advance, is solvable in $O(m \log n)$ time. This chapter studies the min-max version (2.2), where the costs are not known when the tree is chosen: a single spanning tree must be committed to in advance, an adversary then selects the least favourable cost realisation from a prescribed uncertainty set, and the tree is judged by its cost under that worst case.

Section 3.1 writes this objective out at the edge level and examines its structure under each of the three uncertainty models from Section 2.4, isolating the inner worst-case evaluator that drives the subsequent analysis. Section 3.2 treats interval uncertainty, where the problem reduces to a minimum spanning tree under the upper-bound cost vector and is therefore polynomial. Section 3.3 treats discrete uncertainty: weakly NP-hard already at $K = 2$, admitting an FPTAS for any constant K, and strongly NP-hard when K grows with the input, though still approximable within a logarithmic factor in K. Section 3.4 treats budgeted uncertainty, polynomial via a richer reformulation, and a closing summary draws the three models together.

3.1 Problem Formulation

The min-max objective (2.2) introduced in Section 2.4 is stated with the cost function $c(T)$ left unexpanded. We now write it out at the edge level using $c(T) = \sum_{e \in E(T)} c_e$. The resulting *min-max spanning tree problem* is

$$\min_{T \in \mathcal{T}} \max_{c \in \mathcal{U}} \sum_{e \in E(T)} c_e. \tag{3.1}$$

The uncertainty set $\mathcal{U}$ takes one of three forms in this thesis (Definitions 2.1 to 2.3), and the algorithmic character of the problem varies sharply with the choice.

Decision Version and Complexity Classification. The optimisation problem (3.1) asks for both the optimal value and a tree achieving it. The associated *decision version* asks, given a budget $B \in \mathbb{R}$, whether there exists a spanning tree $T \in \mathcal{T}$ with $\max_{c \in \mathcal{U}} c(T) \leq B$. This decision version lies in NP whenever the inner maximisation $\max_{c \in \mathcal{U}} c(T)$ is polynomial-time computable for a fixed tree: the certificate is the tree T, and verification requires confirming $T \in \mathcal{T}$ (an $O(n)$ breadth-first search, as in the deterministic MST decision problem of Section 2.5) and then

evaluating the inner maximum. Under discrete uncertainty this evaluation is the explicit maximum $\max_{k\in\{1,\dots,K\}} c^{(k)}(T)$ over the K scenarios and takes $O(Kn)$ time; under interval and budgeted uncertainty, polynomial inner evaluation follows from the extremal characterisations developed in Sections 3.2 and 3.4. The complexity terminology used in the remainder of the chapter (polynomial time, weak and strong NP-hardness, FPTAS) is the one fixed in Section 2.5.

Worst-Case Evaluator. A central object throughout the chapter is the *worst-case cost* of a fixed spanning tree $T \in \mathcal{T}$:

$$\mathrm{wc}(T) \;:=\; \max_{c\in\mathcal{U}} \sum_{e\in E(T)} c_e. \tag{3.2}$$

With this notation the min-max problem (3.1) reads simply $\min_{T\in\mathcal{T}} \mathrm{wc}(T)$: among all spanning trees, choose the one whose worst-case cost is smallest. How difficult this outer minimisation is depends on the form that $\mathrm{wc}(T)$ takes as a function of the tree T.

The most favourable situation is when $\mathrm{wc}(T)$ is a sum of fixed per-edge weights, that is,

$$\mathrm{wc}(T) = \sum_{e\in E(T)} w_e$$

for some weight vector w that does not itself depend on the choice of T. An objective of this form is called *linear*, and minimising a linear objective over all spanning trees is nothing other than a deterministic minimum spanning tree problem under the weights w, solvable by Kruskal's algorithm in $O(m \log n)$ time. The three uncertainty models differ precisely in whether $\mathrm{wc}(T)$ is linear in this sense.

Under *interval* uncertainty it is. Lemma 3.1 will show that the adversary does best by setting every edge of T to its upper bound, so that $\mathrm{wc}(T) = \sum_{e\in E(T)} u_e$. This is a sum of fixed per-edge weights with $w = u$, and the min-max problem is therefore a minimum spanning tree problem under the upper-bound cost vector u.

Under *discrete* uncertainty it is not. Here $\mathrm{wc}(T) = \max_{k\in\{1,\dots,K\}} c^{(k)}(T)$ is the largest of the K scenario costs of T, that is, the pointwise maximum of the K linear functions $c^{(1)}(T), \dots, c^{(K)}(T)$. Whereas the interval adversary chooses each c_e independently and is captured once and for all by the fixed vector $w = u$, the discrete adversary instead commits to one full scenario covering all edges, and which scenario is worst can depend on the tree T. In general no fixed weight vector w reproduces this tree-dependent choice, so $\mathrm{wc}(T)$ need not collapse into a single sum $\sum_{e\in E(T)} w_e$ across all spanning trees. Consequently the problem does not reduce to one MST computation, and this obstruction is the source of the hardness established in Section 3.3.

Under *budgeted* uncertainty $\mathrm{wc}(T)$ is in general not linear either, because the shared deviation budget couples the edges of T: which of them the adversary drives upward depends on the tree. The problem therefore does not reduce to a single MST computation either, but Section 3.4 shows that it nonetheless remains polynomial.

3.2 Interval Worst-Case Characterisation

We begin with interval uncertainty, the model identified in Section 3.1 as the favourable case. There the worst-case cost $\mathrm{wc}(T)$ requires maximising over the interval set $\mathcal{U} = \prod_{e \in E}[\ell_e, u_e]$, which contains a continuum of cost vectors. The following lemma shows that this maximisation has a transparent solution: confronted with a fixed tree, the adversary simply charges every edge its upper bound.

Lemma 3.1 (Interval Extremal Cost)**.** *Let $\mathcal{U} = \prod_{e \in E}[\ell_e, u_e]$ be an interval uncertainty set. For every spanning tree $T \in \mathcal{T}$, the worst-case cost satisfies*

$$\mathrm{wc}(T) \;=\; \max_{c \in \mathcal{U}} \sum_{e \in E(T)} c_e \;=\; \sum_{e \in E(T)} u_e,$$

and the maximum is attained at the cost vector c^* defined by $c^*_e = u_e$ for every edge $e \in E$.

Proof. Fix a spanning tree $T \in \mathcal{T}$. Only the edges of T appear in the sum $\sum_{e \in E(T)} c_e$, so the costs assigned to edges outside T do not affect the value of the maximisation. Because the uncertainty set $\mathcal{U} = \prod_{e \in E}[\ell_e, u_e]$ is a Cartesian product, the cost of each edge can be chosen from its interval independently of the others. Since each variable c_e appears in exactly one term of the sum, the maximum of the sum equals the sum of the per-edge maxima:

$$\max_{c \in \mathcal{U}} \sum_{e \in E(T)} c_e \;=\; \sum_{e \in E(T)} \max_{c_e \in [\ell_e, u_e]} c_e.$$

Each cost c_e ranges over the interval $[\ell_e, u_e]$ and is maximised at u_e. Substituting yields $\max_{c \in \mathcal{U}} \sum_{e \in E(T)} c_e = \sum_{e \in E(T)} u_e$. The cost vector c^* with $c^*_e = u_e$ for all $e \in E$ lies in $\mathcal{U}$ and attains this value; its entries on edges outside T are immaterial. □

Lemma 3.1 reduces the worst-case cost of a tree to a single sum of upper bounds; this is the linear form anticipated in Section 3.1, with weight vector $w = u$. The consequence for the min-max problem is immediate.

Corollary 3.1 (Polynomial Solvability under Interval Uncertainty)**.** *Under interval uncertainty $\mathcal{U} = \prod_{e \in E}[\ell_e, u_e]$, the min-max spanning tree problem satisfies*

$$\min_{T \in \mathcal{T}} \mathrm{wc}(T) \;=\; \min_{T \in \mathcal{T}} \sum_{e \in E(T)} u_e \;=\; \mathrm{MST}(u),$$

where $\mathrm{MST}(u)$ is the minimum spanning tree cost under the upper-bound cost vector $u = (u_e)_{e \in E}$. The problem is solved by computing a minimum spanning tree under u, which Kruskal's algorithm performs in $O(m \log n)$ time; in particular, it lies in P*.*

Proof. By Lemma 3.1, $\mathrm{wc}(T) = \sum_{e \in E(T)} u_e$ for every $T \in \mathcal{T}$. Minimising over T gives $\min_{T \in \mathcal{T}} \mathrm{wc}(T) = \min_{T \in \mathcal{T}} \sum_{e \in E(T)} u_e$, which is the minimum spanning tree cost $\mathrm{MST}(u)$ by (2.1). Kruskal's algorithm computes a minimum spanning tree under any fixed cost vector in $O(m \log n)$ time (Section 2.3); applied to u, it returns an optimal min-max tree. □

The reduction in Corollary 3.1 is the spanning tree instance of [5, Theorem 4.8], which establishes it for interval min-max problems in general.

Worked Example. We apply Lemma 3.1 to the micro-graph of Figure 2.2, whose upper-bound cost vector is $u = (4, 5, 7, 6, 7)$. By the lemma, the worst-case cost of a tree is the sum of its edges' upper bounds, so for the three representative trees

$$\mathrm{wc}(T_1) = 4+5+7 = 16, \qquad \mathrm{wc}(T_2) = 4+5+6 = 15, \qquad \mathrm{wc}(T_3) = 5+7+7 = 19.$$

These values coincide with the scenario-$c^{(2)}$ column of Table 2.1, as they must: the micro-graph's scenario $c^{(2)}$ was defined as the all-upper-bounds vector, so $c^{(2)} = u$.

To find the min-max spanning tree, Corollary 3.1 directs us to a minimum spanning tree under u. Kruskal's algorithm processes the edges in non-decreasing order of u, namely e_1 (4), e_2 (5), e_4 (6), followed by e_3 and e_5 (both 7). It adds $e_1 = \{1, 2\}$, $e_2 = \{2, 3\}$ and $e_4 = \{3, 4\}$; the tree is then complete, and the two remaining edges are not examined. The result is $T_2 = \{e_1, e_2, e_4\}$ with cost $\mathrm{MST}(u) = 15$. The three trees evaluated above are only three of the micro-graph's eight spanning trees; what certifies T_2 as the min-max tree over all of them is Corollary 3.1, applied through this minimum spanning tree computation, rather than the three-way comparison alone. The entries of u are not all distinct, e_3 and e_5 sharing the upper bound 7, so the sufficient condition of Remark 2.1 does not apply here. The minimum is nonetheless attained by T_2 alone: the seven remaining spanning trees have u-costs between 16 and 20. Hence T_2 is the unique min-max spanning tree, its cost matching the value $\mathrm{wc}(T_2) = 15$ computed above.

The Role of the Lower Bounds. Lemma 3.1 shows that the min-max objective depends on the interval data only through the upper bounds u; the lower bounds ℓ play no role, and the entire interval model compresses to the single cost vector u. Two points are worth drawing out. First, the separation step in the proof used only that $\mathcal{U}$ is a product of intervals; it is exactly this absence of coupling between edges that fails under budgeted uncertainty (Section 3.4), where a shared deviation budget links the edges and the worst case is in general no longer a fixed per-edge sum. Second, the insensitivity to the lower bounds is special to the min-max objective: the regret objective of Chapter 4 measures each tree against the scenario-optimal tree, a comparison that does depend on the lower bounds, and there the interval problem becomes substantially harder.

3.3 Complexity and Approximation

Under interval uncertainty the entire model compressed into the single cost vector u, and the min-max problem collapsed to one minimum spanning tree computation (Lemma 3.1 and Corollary 3.1). Discrete uncertainty admits no such compression: by (3.2), the worst-case cost is the largest of the K scenario costs of the tree, and the maximising scenario may change from tree to tree (Section 3.1). This section turns that obstruction into complexity results, and the price depends on whether the number of scenarios is a fixed constant or part of the input. For every constant K the problem is weakly NP-hard, already for $K = 2$; this is Theorem 3.1, the hardness result of this chapter that we prove in full, complemented by a pseudo-polynomial algorithm and an FPTAS (Theorems 3.2 and 3.3). When K is part of the input, the hardness becomes strong and no constant-factor approximation exists at all, the best guarantee being an $O(\log K)$-approximation; these results close the section.

3.3.1 Discrete Scenarios: Constant K

Throughout this subsection the number of scenarios K is a constant, fixed independently of the instance.

Theorem 3.1 (Hardness for Two Scenarios)**.** *Under discrete uncertainty, the min-max spanning tree problem is weakly* NP*-hard, even when restricted to $K = 2$ scenarios.*

Theorem 3.1 is due to Kouvelis and Yu [1]; the presentation here follows Goerigk and Hartisch [5, Theorem 8.4]. The reduction starts from the *partition problem*, which is NP-complete [2, Corollary 15.28]: given positive integers $w_1, \dots, w_n$, decide whether some subset $P \subseteq \{1, \dots, n\}$ satisfies $\sum_{i \in P} w_i = \frac{1}{2} \sum_{i=1}^{n} w_i$. Throughout, $W = \sum_{i=1}^{n} w_i$ denotes the total weight and $Q = W/2$ the target value.

From a partition instance we build the grid graph $G = (V, E)$ shown in Figure 3.1, with two rows and $n + 1$ columns. Its vertex set is

$$V = \{v_{i,j} : i \in \{1, 2\},\ j \in \{0, 1, \dots, n\}\},$$

and two vertices are adjacent exactly when they agree in one index and differ by one in the other, that is,

$$\{v_{i,j},\, v_{k,\ell}\} \in E \quad \Longleftrightarrow \quad \Big(i = k \text{ and } |j - \ell| = 1\Big) \ \text{ or } \ \Big(|i - k| = 1 \text{ and } j = \ell\Big).$$

For $j \in \{1, \dots, n\}$ we write $a_j = \{v_{1,j-1}, v_{1,j}\}$ for the *first-row edge of column j* and $b_j = \{v_{2,j-1}, v_{2,j}\}$ for the *second-row edge of column j*; together these $2n$ edges are the *horizontal edges*, each named after the column of its right endpoint. The remaining $n + 1$ edges are the *vertical edges* $r_j = \{v_{1,j}, v_{2,j}\}$, one for every column $j \in \{0, 1, \dots, n\}$. The two scenarios place each weight once in each row:

$$c^{(1)}_{a_j} = c^{(2)}_{b_j} = w_j, \qquad c^{(1)}_{b_j} = c^{(2)}_{a_j} = 0 \qquad \text{for } j \in \{1, \dots, n\},$$

and $c^{(1)}_{r_j} = c^{(2)}_{r_j} = 0$ for all $j \in \{0, 1, \dots, n\}$.

The intended mechanism is visible in Figure 3.1. A spanning tree must pass from column $j - 1$ to column j through a_j or b_j, paying w_j in scenario 1 or in scenario 2 respectively, so choosing a tree distributes the weights $w_1, \dots, w_n$ between the two scenarios; the worst case is the larger of the two shares, smallest when the split is even. The proof makes this correspondence exact.

Proof of Theorem 3.1. The construction is computable in polynomial time: G has $2(n + 1)$ vertices and $3n + 1$ edges, and every cost is zero or one of the given weights. We claim that G admits a spanning tree of worst-case cost at most Q if and only if the partition instance is a yes-instance; this is the decision version of Section 3.1 with budget $B = Q$.

Step 1: some optimal tree contains every vertical edge. Suppose a spanning tree T does not contain the vertical edge r_j of some column j. Adding r_j to T creates exactly one cycle (property P3), the fundamental cycle C_{r_j}, consisting of r_j and the unique T-path from $v_{1,j}$ to $v_{2,j}$. This path connects the two rows, so it crosses between them along some vertical edge of T; as $r_j \notin E(T)$, that edge is some r_ℓ

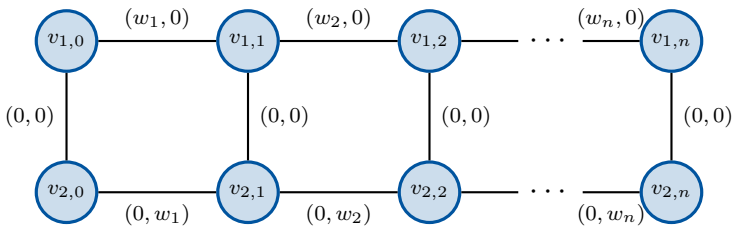

Figure 3.1: The grid graph G built from a partition instance with weights $w_1, \ldots, w_n$. Each edge is labelled with its cost pair $(c_e^{(1)}, c_e^{(2)})$: a first-row edge of column j costs w_j in scenario 1, a second-row edge of column j costs w_j in scenario 2, and every vertical edge costs zero in both scenarios.

with $\ell \neq j$, and in travelling between columns j and ℓ the path uses at least one horizontal edge h. The exchange $T' = (T \setminus \{h\}) \cup \{r_j\}$ is again a spanning tree: deleting h breaks the unique cycle of $T \cup \{r_j\}$, while the remainder of C_{r_j} keeps the endpoints of h connected. Since $c_{r_j}^{(k)} = 0 \leq c_h^{(k)}$ for $k \in \{1, 2\}$, neither scenario cost increases, and $\mathrm{wc}(T') \leq \mathrm{wc}(T)$. Each exchange raises the number of vertical edges by one, so iterating turns any spanning tree into one that contains all vertical edges at no larger worst-case cost; in particular, the minimum worst-case cost is attained by such a tree.

Step 2: with the vertical edges fixed, a spanning tree is just a choice of one row per column, and its worst-case cost depends on that choice alone. With every vertical edge present, the two vertices of each column are already joined, and a spanning tree's only remaining decision is, for each column j, whether it crosses from column $j-1$ through the first-row edge a_j or the second-row edge b_j. For a subset $P \subseteq \{1, \ldots, n\}$, let T_P consist of every vertical edge together with the first-row edge a_j in the columns $j \in P$ and the second-row edge b_j in the rest:

$$T_P := \{r_0, r_1, \ldots, r_n\} \cup \{a_j : j \in P\} \cup \{b_j : j \notin P\}.$$

We show that the trees containing every vertical edge are exactly the sets T_P. Each T_P is a spanning tree: the vertical edges join the two rows within every column and the n chosen horizontal edges link consecutive columns, so T_P is connected and, with its $(n+1) + n = |V| - 1$ edges, is a tree. Conversely, let T be a spanning tree containing every vertical edge, and fix $j \in \{1, \ldots, n\}$. Let $V_{<j} := \{v_{i,\ell} : i \in \{1, 2\},\ \ell \in \{0, 1, \ldots, j-1\}\}$ be the vertices of the first j columns. The cut $\delta(V_{<j}) = \{a_j, b_j\}$ consists of the only two edges straddling the boundary between columns $j-1$ and j (Figure 3.1), so connectivity forces at least one of a_j, b_j into $E(T)$, while $r_{j-1}, r_j \in E(T)$ forbid both, since together they would close the four-cycle a_j, r_j, b_j, r_{j-1}. Thus T contains exactly one edge of each pair $\{a_j, b_j\}$, and as G has only vertical and horizontal edges, $T = T_P$ with $P = \{j : a_j \in E(T)\}$. The worst-case cost now follows: scenario 1 charges only first-row edges and scenario 2 only second-row edges, so with $w(P) = \sum_{j \in P} w_j$,

$$c^{(1)}(T_P) = w(P), \qquad c^{(2)}(T_P) = W - w(P), \qquad \mathrm{wc}(T_P) = \max\{w(P),\, W - w(P)\}.$$

Step 3: a tree of worst-case cost at most Q exists if and only if the weights split evenly. By Steps 1 and 2 the minimisation over trees becomes a minimisation over subsets,

$$\min_{T \in \mathcal{T}} \mathrm{wc}(T) \;=\; \min_{P \subseteq \{1,\dots,n\}} \max\{w(P),\, W - w(P)\}.$$

Since $w(P)$ and $W - w(P)$ sum to $2Q$, their maximum is at least Q, with equality if and only if $w(P) = Q$. The minimum worst-case cost is therefore at least Q, and equals Q exactly when some subset satisfies $w(P) = Q$, which is the partition condition. As the minimum never falls below Q, a spanning tree of worst-case cost at most Q exists if and only if the partition instance is a yes-instance, proving the claim.

The partition problem therefore reduces in polynomial time to the decision version of the min-max spanning tree problem with $K = 2$, and the problem is NP-hard. Since Theorem 3.2 provides a pseudo-polynomial algorithm, the hardness is weak in the sense of Section 2.5. □

The restriction to $K = 2$ is the sharpest form of the result rather than a limitation: for any constant $K \geq 2$, duplicating one of the two scenarios $K - 2$ times changes no worst-case cost, so the problem with exactly K scenarios is weakly NP-hard as well.

A Small Instance. Consider the weights $w = (3, 1, 2)$, so $n = 3$, $W = 6$ and $Q = 3$. The subset $P = \{1\}$ satisfies $w(P) = 3 = Q$, and the tree from the proof,

$$T_P = \{r_0, r_1, r_2, r_3\} \cup \{a_1, b_2, b_3\},$$

has $c^{(1)}(T_P) = w_1 = 3$ and $c^{(2)}(T_P) = w_2 + w_3 = 3$, hence $\mathrm{wc}(T_P) = 3 = Q$; by Step 3 no spanning tree does better, and the reduction reports a yes-instance. The weights $w = (1, 5)$ give the same $W = 6$ and $Q = 3$, but the four subsets $P \subseteq \{1, 2\}$ yield $w(P) \in \{0, 1, 5, 6\}$ and worst-case costs $\max\{w(P),\, W - w(P)\} \in \{5, 6\}$; the optimum is $5 > Q$, so no spanning tree meets the threshold, matching the fact that no subset of $\{1, 5\}$ sums to 3.

Theorem 3.1 proves the problem hard, but its reduction reveals the hardness to be of a mild and familiar kind. All of the difficulty came from splitting a set of weights into two equal halves, the partition problem. None came from the spanning tree structure, which merely carried the two scenarios. Hardness of this numerical kind is weak: it bites only when the numbers are large and eases once they are kept small. The knapsack problem of Section 2.5 is the canonical example, NP-hard yet admitting an FPTAS. For a constant number of scenarios, the min-max spanning tree problem follows the same pattern.

Theorem 3.2 (Pseudo-Polynomial Algorithm)**.** *For every constant K, the min-max spanning tree problem under discrete uncertainty is solvable in pseudo-polynomial time.*

Theorem 3.3 (FPTAS)**.** *For every constant K, the min-max spanning tree problem under discrete uncertainty admits a fully polynomial-time approximation scheme.*

Both algorithms are built on one subroutine, the *exact spanning tree problem*: given integer edge costs and a target value, does some spanning tree have exactly that total cost? This is solvable in pseudo-polynomial time. Its running time grows with the magnitude of the costs, not with the size of the graph alone [7].

The subroutine turns the min-max problem into a search. Under K scenarios a tree carries a list of K costs, one per scenario, and its worst-case cost is the largest entry of that list. The min-max optimum is the tree whose largest entry is smallest. Each of the K entries is an integer between zero and the total cost of all edges in its own scenario, so when K is fixed only pseudo-polynomially many lists can occur. Searching these few lists replaces the search over the exponentially many spanning trees. Bundling each edge's K costs into a single number, in a base wide enough that the scenarios cannot interfere, lets the subroutine test whether a prescribed list is realised by some tree. Iterating it over the candidate lists and keeping the realised one of smallest maximum returns the min-max optimum in pseudo-polynomial time [5, Theorem 8.2].

The approximation scheme runs the same search on scaled costs. Dividing every cost by a common factor, chosen from ε and a bound on the optimum, and then rounding shrinks the magnitudes until only polynomially many lists remain. This turns the pseudo-polynomial search into a polynomial one, while distorting each scenario cost by at most a factor $1 + \varepsilon$. The returned tree therefore has worst-case cost within $1 + \varepsilon$ of the optimum, in time polynomial in the input and in $1/\varepsilon$ [5, Theorem 8.3]. This scheme is due to Aissi, Bazgan and Vanderpooten [8].

These results supply the pseudo-polynomial algorithm that Theorem 3.1's proof invoked to classify its hardness as weak. With an FPTAS available as well, the problem cannot be strongly hard unless $\mathsf{P} = \mathsf{NP}$, since a strongly hard problem would admit neither. Both guarantees hold only because K is fixed, which keeps the number of cost lists small. This is exactly what the next subsection removes: once K grows with the input, the lists are no longer few, and both the algorithm and the approximation scheme fall away.

3.3.2 Discrete Scenarios: Unbounded K

With K part of the input, the hardness of the problem changes in kind. For a fixed number of scenarios the difficulty was numerical, the partition problem in disguise, and that is why it was weak and yielded to scaling. With arbitrarily many scenarios available, a purely logical problem can be encoded instead, one that involves no large numbers at all, and the problem becomes strongly hard and far harder to approximate.

Theorem 3.4 (Strong NP-Hardness and Inapproximability)**.** *If K is part of the input, the min-max spanning tree problem under discrete uncertainty is strongly* NP*-hard, and unless* $\mathsf{P} = \mathsf{NP}$ *it cannot be approximated within a factor of* $2 - \varepsilon$ *for any* $\varepsilon > 0$*.*

The result is due to Kasperski and Zieliński [9]; the presentation here follows Goerigk and Hartisch [5, Theorem 8.5]. The reduction starts from 3-SAT, which is NP-complete [2, Theorem 15.22]: given a Boolean formula whose clauses each contain three literals, each a variable or its negation, decide whether some truth assignment satisfies every clause.

The graph receives a small gadget for each clause, shown in Figure 3.2, and the gadgets are chained one after another, so that every route from one end of the graph to the other must pass through each gadget in turn. Inside the gadget of clause i lie three parallel two-edge routes from a_i to b_i; the first edge $\{a_i, v_{i,j}\}$ of the j-th route represents the j-th literal $L_{i,j}$ of the clause, while the second edge, like the edges linking consecutive gadgets, costs zero in every scenario. Crossing the gadget requires at least one of its literal edges, so every spanning tree selects at least one literal in every clause, and the only costs a tree can ever incur sit on the literal edges it selects. The scenarios then encode the logic: for every complementary pair of occurrences, $L_{i,j} = \neg L_{i',j'}$, a scenario is added that charges cost 1 to the two edges $\{a_i, v_{i,j}\}$ and $\{a_{i'}, v_{i',j'}\}$ and 0 to every other edge.

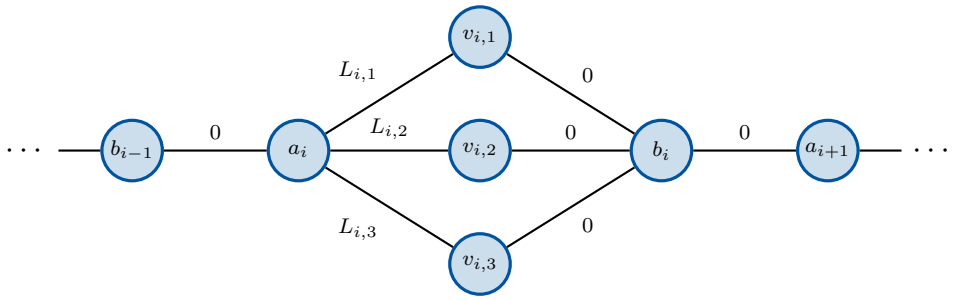

Figure 3.2: The gadget of clause i in the reduction from 3-SAT, chained to its neighbouring gadgets through b_{i-1} and a_{i+1}. The edge $\{a_i, v_{i,j}\}$ carries the j-th literal $L_{i,j}$ of the clause; a scenario charges cost 1 to the two literal edges of a complementary pair of occurrences and 0 everywhere else. All edges labelled 0 are free in every scenario, so a tree's cost arises only on the literal edges it selects, and crossing the gadget from a_i to b_i forces at least one of them.

The correspondence is exact in both directions. If the formula is satisfiable, fix a satisfying assignment and build the tree that takes, in each clause, one literal edge whose literal the assignment makes true, together with all the cost-free edges. Each gadget contributes five vertices and four selected edges, with one connector joining each consecutive pair, so the selection has one edge fewer than the graph has vertices; being connected as well, it is a spanning tree. No two selected literals are complementary, both being true under one assignment, so every scenario charges at most one selected edge, and the worst-case cost is at most 1. Conversely, suppose some spanning tree avoids both edges of every complementary pair. Setting each of its selected literals to true is then consistent, and since every clause contributes a selected literal, the formula is satisfied. For an unsatisfiable formula every spanning tree therefore contains both edges of some pair, and the scenario of that pair costs it 2; since no scenario has more than two unit-cost edges, no tree costs more than 2 in any scenario, and the optimal worst-case cost is exactly 2.

The optimal worst-case cost is thus at most 1 for a satisfiable formula and exactly 2 for an unsatisfiable one, so deciding which of the two occurs decides 3-SAT, and the problem is NP-hard. Every cost in the construction is 0 or 1, so the numbers remain polynomially bounded and the hardness is strong, in contrast to the partition reduction, where large weights were essential.

The same gap yields the inapproximability bound. Suppose that, for some $\varepsilon > 0$, a polynomial-time algorithm always returned a spanning tree of worst-case cost at most $2 - \varepsilon$ times the optimum. On a satisfiable instance the optimum is at most 1, so the returned tree would cost at most $2 - \varepsilon < 2$; on an unsatisfiable instance every tree costs exactly 2, the returned one included. Reading off whether the returned tree costs less than 2 would therefore decide 3-SAT in polynomial time, which is impossible unless $\mathsf{P} = \mathsf{NP}$. The inapproximability is in fact far stronger, no constant factor being achievable at all [10]; the self-contained $2 - \varepsilon$ bound above is its modest form.

Theorem 3.5 (Logarithmic Approximation)**.** *The min-max spanning tree problem under discrete uncertainty can be approximated within a factor of* $O(\log K)$*.*

This closes the gap from above, leaving only a logarithmic distance to the no-constant-factor barrier. The obstacle is the worst-case objective itself: a maximum over the K scenario costs is far harder to minimise than the plain sum that reduced the interval problem to a single spanning tree computation (Corollary 3.1). The remedy is to minimise a sum after all, one built to imitate the maximum. For a parameter $p \geq 1$, aggregate the K entries of a tree's cost list into the p-th root of the sum of their p-th powers, in the literature the p-norm of the list. Since the scenario costs are non-negative (Definition 2.1), the aggregate brackets the worst case,

$$\mathrm{wc}(T) \;\leq\; \Bigg(\sum_{k=1}^{K}\Big(c^{(k)}(T)\Big)^{p}\Bigg)^{1/p} \;\leq\; K^{1/p}\,\mathrm{wc}(T), \tag{3.3}$$

the first inequality because the sum contains the p-th power of its largest entry, the second because it consists of K terms, none larger. The larger p is, the closer the factor $K^{1/p}$ draws to one and the more faithfully the aggregate tracks the maximum.

Unlike the maximum, the aggregate is an objective a greedy method can handle. Growing the tree edge by edge in the manner of Kruskal's algorithm, but adding at each step, among the edges creating no cycle, one that increases the aggregate the least, returns a spanning tree whose aggregate is at most $p/\ln 2$ times the smallest possible [5, Theorem 5.21]. The two error sources now pull against each other: raising p tightens the right-hand inequality of (3.3) but loosens the greedy guarantee. Both are paid in succession. By the left inequality of (3.3) the greedy tree's worst-case cost is at most its aggregate, and that aggregate is at most $p/\ln 2$ times the smallest aggregate any spanning tree attains, in particular at most $p/\ln 2$ times the aggregate of a min-max optimal tree. By the right inequality, that last aggregate is at most $K^{1/p}$ times the min-max optimum. The greedy tree therefore lies within a factor $(p/\ln 2)\cdot K^{1/p}$ of that optimum. Choosing $p = \ln K$ makes both factors explicit: writing $K = e^{\ln K}$ gives $K^{1/p} = K^{1/\ln K} = e$, while $p/\ln 2 = \ln K/\ln 2 = \log_2 K$, so that

$$\frac{p}{\ln 2}\cdot K^{1/p} \;=\; e\log_2 K \;=\; O(\log K),$$

which is the guarantee of Theorem 3.5 [5, Corollary 5.22]. The bound is due to Baak et al. [11], building on the greedy analysis of Bilò et al. [12].

The discrete-uncertainty picture is now complete, and the number of scenarios is its dividing line. For a fixed number of scenarios the problem is only weakly hard and

admits an FPTAS; once that number is part of the input, it becomes strongly hard, approximable within $O(\log K)$ but within no constant factor. One problem thus carries two kinds of hardness: a numerical one that scaling defeats, and a structural one that it cannot touch.

3.4 Budgeted Uncertainty

The last of the three uncertainty models restores polynomial-time solvability, though by a longer route than interval uncertainty required. In the budgeted set $\mathcal{U}^\Gamma$ of Definition 2.3, each edge may rise from its nominal cost $\bar{c}_e$ by at most its maximum deviation $\hat{c}_e$, but a single budget caps how much deviation the adversary may distribute in total, $\sum_{e\in E} \delta_e \le \Gamma$. That shared budget couples the edges of the chosen tree, as Section 3.1 observed. The maximisation therefore no longer separates edge by edge as it did in Section 3.2, because budget spent on one edge is budget another cannot have, and which edges the adversary drives upward depends on the tree. In general no fixed weight vector reproduces $\mathrm{wc}(T)$, and the collapse to a single minimum spanning tree that settled the interval case (Corollary 3.1) is unavailable. What replaces it is a richer reformulation. The adversary's tree-dependent choice can be absorbed into a single parameter, though this means solving not one deterministic minimum spanning tree problem but a small family of them, one for each candidate value of that parameter. As in Section 3.2, we begin with a fixed tree and turn to the minimisation over all trees afterwards.

Worst-Case Cost of a Fixed Tree. Fix a spanning tree $T \in \mathcal{T}$. Evaluating (3.2) over $\mathcal{U}^\Gamma$ confronts the adversary with a choice its interval counterpart never faced: unless the budget covers the whole tree, it cannot raise every edge of T at once and must decide where a limited allowance does the most damage. The choice is nevertheless an easy one, namely to spend the budget on the edges of T carrying the largest deviations, raising each of them in full. The following lemma confirms that this greedy rule is optimal and evaluates the worst case in closed form.

Lemma 3.2 (Budgeted Extremal Cost)**.** *Let $\mathcal{U}^\Gamma$ be a budgeted uncertainty set with nominal costs $\bar{c}$, maximum deviations $\hat{c}$, and an integer deviation budget $\Gamma \ge 0$. Let $T \in \mathcal{T}$ be a spanning tree, order its edges as $E(T) = \{f_1, \dots, f_{n-1}\}$ so that $\hat{c}_{f_1} \ge \hat{c}_{f_2} \ge \cdots \ge \hat{c}_{f_{n-1}}$, and set $g := \min\{\Gamma, n-1\}$. Then the worst-case cost of T satisfies*

$$\mathrm{wc}(T) \;=\; \sum_{e\in E(T)} \bar{c}_e \;+\; \sum_{j=1}^{g} \hat{c}_{f_j},$$

and the maximum in (3.2) is attained at the cost vector c^* given by $c^*_{f_j} = \bar{c}_{f_j} + \hat{c}_{f_j}$ for $j \le g$ and $c^*_e = \bar{c}_e$ for every other edge $e \in E$.

Proof. Every $c \in \mathcal{U}^\Gamma$ has the form $c_e = \bar{c}_e + \hat{c}_e\delta_e$ with $\delta \in [0,1]^{|E|}$ and $\sum_{e\in E} \delta_e \le \Gamma$ (Definition 2.3). The nominal part of $c(T)$ is the same for every scenario, so only the deviations are at stake and (3.2) becomes

$$\mathrm{wc}(T) \;=\; \sum_{e\in E(T)} \bar{c}_e \;+\; \max\left\{ \sum_{e\in E(T)} \hat{c}_e\delta_e \;:\; \delta \in [0,1]^{|E|},\; \sum_{e\in E} \delta_e \le \Gamma \right\}. \tag{3.4}$$

It suffices to show that this maximum equals $\sum_{j=1}^{g} \hat{c}_{f_j}$ and is attained at the deviation vector realising c^*.

The objective in (3.4) involves only the edges of T. Setting $\delta_e := 0$ for every $e \notin E(T)$ therefore leaves the objective unchanged while freeing budget, so some maximiser vanishes outside $E(T)$, and for such δ the budget constraint reads $\sum_{j=1}^{n-1} \delta_{f_j} \leq \Gamma$.

If $g = 0$, then either $\Gamma = 0$, so that the budget forces every δ_e to zero, or T has no edges at all; in both cases the maximum is the empty sum, as claimed. Assume from now on that $g \geq 1$.

The candidate maximiser is the vector δ^* with $\delta^*_{f_j} = 1$ for $j \leq g$ and $\delta^*_e = 0$ for every other edge, which realises exactly the cost vector c^*. Its total deviation is $g \leq \Gamma$, so δ^* is feasible, and its objective value is $\sum_{j=1}^{g} \hat{c}_{f_j}$. The maximum is therefore at least this value, and it remains to show that no feasible δ exceeds it.

Let δ be feasible and vanishing outside $E(T)$. Comparing its value with that of δ^* and grouping the terms according to whether $j \leq g$,

$$\sum_{j=1}^{n-1} \hat{c}_{f_j}\, \delta_{f_j} \;-\; \sum_{j=1}^{g} \hat{c}_{f_j} \;=\; \sum_{j=1}^{g} \hat{c}_{f_j}\big(\delta_{f_j} - 1\big) \;+\; \sum_{j=g+1}^{n-1} \hat{c}_{f_j}\, \delta_{f_j}.$$

We bound the right-hand side by replacing every coefficient $\hat{c}_{f_j}$ with the single value $\hat{c}_{f_g}$, the smallest deviation that δ^* pays for. By the chosen ordering this replacement can only increase the terms, in each sum for its own reason. For $j \leq g$ the factor $\delta_{f_j} - 1$ is non-positive while $\hat{c}_{f_j} \geq \hat{c}_{f_g}$, and scaling a non-positive quantity by a smaller coefficient increases it. For $j > g$ the factor δ_{f_j} is non-negative while $\hat{c}_{f_j} \leq \hat{c}_{f_g}$, and scaling a non-negative quantity by a larger coefficient increases it. Both sums then carry $\hat{c}_{f_g}$ as a common factor, so

$$\sum_{j=1}^{n-1} \hat{c}_{f_j}\, \delta_{f_j} \;-\; \sum_{j=1}^{g} \hat{c}_{f_j} \;\leq\; \hat{c}_{f_g} \left(\sum_{j=1}^{n-1} \delta_{f_j} \;-\; g \right).$$

The bracket is never positive. If $\Gamma \leq n-1$, then $g = \Gamma$ and the budget constraint gives $\sum_{j=1}^{n-1} \delta_{f_j} \leq \Gamma = g$; if $\Gamma > n-1$, then $g = n-1$ and the bounds $\delta_{f_j} \leq 1$ alone give $\sum_{j=1}^{n-1} \delta_{f_j} \leq n-1 = g$. Since $\hat{c}_{f_g} \geq 0$ by Definition 2.3, the whole right-hand side is at most zero, so δ does not beat δ^*. The maximum in (3.4) is therefore $\sum_{j=1}^{g} \hat{c}_{f_j}$, attained at δ^*. □

The lemma assumes an integer budget, whereas Definition 2.3 allows any $\Gamma \in [0, |E|]$. A fractional budget changes little: the adversary fills the largest deviations in the same order, and any budget left over once whole edges are exhausted is spent on the next largest deviation, which adds at most one fractional term to the closed form. Every budget appearing in what follows is an integer.

Evaluating $\mathrm{wc}(T)$ is now a sorting task. Ordering the $n-1$ deviations of $E(T)$ and adding the g largest to the nominal cost of T takes $O(n \log n)$ time, which is the polynomial inner evaluation promised for the budgeted model in Section 3.1.

The formula also displays both ends of the budget range. At $\Gamma = 0$ it returns $\mathrm{wc}(T) = \sum_{e \in E(T)} \bar{c}_e$, so the min-max problem is the deterministic minimum spanning tree problem under the nominal costs $\bar{c}$. Once $\Gamma \geq n-1$ we have $g = n-1$, every edge of T deviates fully, and $\mathrm{wc}(T) = \sum_{e \in E(T)} (\bar{c}_e + \hat{c}_e)$ is the interval worst case of

Lemma 3.1 on the box $\prod_{e\in E}[\bar{c}_e,\ \bar{c}_e+\hat{c}_e]$. Definition 2.3 reaches that box at $\Gamma=|E|$, but against spanning trees the interval behaviour sets in earlier, at $\Gamma=n-1$: a tree offers the adversary only $n-1$ edges to spend on, and any further budget is wasted. Across the whole range $\mathrm{wc}(T)$ is non-decreasing in Γ, since enlarging the budget enlarges $\mathcal{U}^{\Gamma}$. At both ends it is once more a sum of fixed per-edge weights, with $w=\bar{c}$ and $w=\bar{c}+\hat{c}$ respectively. Only for budgets strictly between them can the coupling described above take effect.

Minimisation over All Trees. Lemma 3.2 selects deviations by rank, and rank is what stands in the way of minimising over all trees. Whether an edge belongs to the g largest is a question about the other edges of its tree as much as about the edge itself, so the selection moves with T and cannot be written down edge by edge.

Rank can be traded for a threshold. Fix a level $\pi\geq 0$ and charge each edge only the part of its deviation that exceeds it, namely $[\hat{c}_e-\pi]^+$ with $[z]^+:=\max\{z,0\}$, a quantity the edge settles on its own. An edge whose deviation reaches the level is charged exactly π less than that deviation, so a flat payment of $\Gamma\pi$ can make good Γ such shortfalls at once. Placing the level at the Γ-th largest deviation of T, for $1\leq\Gamma\leq n-1$, balances the account: the Γ largest deviations all reach that level and one copy of π restores each of them in full, while the remaining edges do not exceed it and are charged nothing. The account then totals exactly the worst-case cost of Lemma 3.2, and the following lemma shows that no level does better.

Lemma 3.3 (Threshold Form of the Worst-Case Cost)**.** *Let $\mathcal{U}^{\Gamma}$ be a budgeted uncertainty set with an integer deviation budget $\Gamma\geq 0$. Every spanning tree $T\in\mathcal{T}$ then satisfies*

$$\mathrm{wc}(T)\ =\ \min_{\pi\geq 0}\left[\Gamma\pi\ +\ \sum_{e\in E(T)}\Big(\bar{c}_e+[\hat{c}_e-\pi]^+\Big)\right],$$

and the minimum is attained at some $\pi\in\{0\}\cup\{\hat{c}_e : e\in E(T)\}$.

Proof. Order $E(T)=\{f_1,\dots,f_{n-1}\}$ by non-increasing deviations as in Lemma 3.2, and put $g:=\min\{\Gamma,\ n-1\}$. The nominal costs do not involve π and move outside the minimisation, so by Lemma 3.2 the claim is equivalent to

$$\min_{\pi\geq 0}\varphi_T(\pi)\ =\ \sum_{j=1}^{g}\hat{c}_{f_j}\qquad\text{for}\quad\varphi_T(\pi)\ :=\ \Gamma\pi+\sum_{j=1}^{n-1}\Big[\hat{c}_{f_j}-\pi\Big]^+,$$

with a minimiser among 0 and the deviations of T. We treat three ranges of the budget; in each we exhibit a level of the required form that attains the claimed value, and then verify that no level falls below it.

Suppose first that $\Gamma=0$, so that $g=0$ and the claimed value is the empty sum. If T has no edges, then φ_T vanishes identically; otherwise the level $\pi=\hat{c}_{f_1}$ makes every bracket vanish, giving $\varphi_T(\hat{c}_{f_1})=0$. No level falls below this, because every summand of φ_T is non-negative.

Suppose next that $1\leq\Gamma\leq n-1$, so that $g=\Gamma$. At the level $\pi=\hat{c}_{f_g}$ the ordering gives $[\hat{c}_{f_j}-\hat{c}_{f_g}]^+=\hat{c}_{f_j}-\hat{c}_{f_g}$ for $j<g$ and $[\hat{c}_{f_j}-\hat{c}_{f_g}]^+=0$ for $j\geq g$, so that

$$\varphi_T\Big(\hat{c}_{f_g}\Big)\ =\ g\,\hat{c}_{f_g}\ +\ \sum_{j=1}^{g-1}\Big(\hat{c}_{f_j}-\hat{c}_{f_g}\Big)\ =\ \sum_{j=1}^{g}\hat{c}_{f_j},$$

the flat payment restoring each of the g largest deviations to its full value. No level falls below this value either, as two elementary bounds show. Let $\pi \geq 0$. The summands of φ_T with $j > g$ are non-negative and may be dropped, and each remaining bracket satisfies $[z]^+ \geq z$, so

$$\varphi_T(\pi) \;\geq\; \Gamma\pi + \sum_{j=1}^{g} \Big[\hat{c}_{f_j} - \pi\Big]^+ \;\geq\; \Gamma\pi + \sum_{j=1}^{g} \Big(\hat{c}_{f_j} - \pi\Big) \;=\; \sum_{j=1}^{g} \hat{c}_{f_j},$$

the final equality because $\Gamma = g$, so that the g subtracted copies of π cancel the flat payment.

Suppose finally that $\Gamma > n-1$, so that $g = n-1$. At the level $\pi = 0$ every bracket returns its deviation unchanged, giving $\varphi_T(0) = \sum_{j=1}^{n-1} \hat{c}_{f_j}$. For any $\pi \geq 0$, applying $[z]^+ \geq z$ to all $n-1$ summands,

$$\varphi_T(\pi) \;\geq\; \Gamma\pi + \sum_{j=1}^{n-1} \Big(\hat{c}_{f_j} - \pi\Big) \;=\; \sum_{j=1}^{n-1} \hat{c}_{f_j} + \Big(\Gamma - (n-1)\Big)\pi,$$

and the final term is non-negative because $\Gamma > n-1$ and $\pi \geq 0$.

The three ranges partition the integer budgets $\Gamma \geq 0$, and in each the exhibited level lies in $\{0\} \cup \{\hat{c}_e : e \in E(T)\}$, which proves the lemma. □

The attaining level need not be unique: at $\Gamma = n-1$ the minimum is attained both at $\pi = \hat{c}_{f_{n-1}}$ and at $\pi = 0$, where the brackets already return every deviation in full. For the minimisation over all trees this does not matter; what matters is that some minimiser lies in a set fixed in advance. The candidate set of Lemma 3.3 still depends on T, but enlarging it to the deviations of all edges makes it the same for every tree, and the two minimisations can then be interchanged.

Theorem 3.6 (Polynomial Solvability under Budgeted Uncertainty)**.** *Let $\mathcal{U}^\Gamma$ be a budgeted uncertainty set with an integer deviation budget $\Gamma \geq 0$, put $\Pi := \{0\} \cup \{\hat{c}_e : e \in E\}$, and for $\pi \in \Pi$ let c^π be the cost vector with $c^\pi_e := \bar{c}_e + [\hat{c}_e - \pi]^+$. Then the min-max spanning tree problem under budgeted uncertainty satisfies*

$$\min_{T \in \mathcal{T}} \mathrm{wc}(T) \;=\; \min_{\pi \in \Pi} \Big[\, \Gamma\pi + \mathrm{MST}(c^\pi) \Big],$$

and if π^* attains the right-hand minimum, then every minimum spanning tree under c^{π^*} is min-max optimal. The problem is solvable in $O(m^2 \log n)$ time; in particular, it lies in P*.*

The result is due to Bertsimas and Sim [13], who prove it for min-max problems over an arbitrary set of binary solutions; the presentation here follows Goerigk and Hartisch [5, Theorem 4.21], in whose derivation the threshold form of Lemma 3.3 appears as an intermediate reformulation.

Proof. For every $T \in \mathcal{T}$ the set Π contains $\{0\} \cup \{\hat{c}_e : e \in E(T)\}$ and therefore, by Lemma 3.3, a minimiser of the threshold form; enlarging the candidate set in this way

is what removes its dependence on T [5, Lemma 4.20]. Restricting a minimisation to a subset containing a minimiser leaves its value unchanged, so

$$\mathrm{wc}(T) \;=\; \min_{\pi\in\Pi}\left[\Gamma\pi + \sum_{e\in E(T)} c_e^{\pi}\right] \qquad \text{for every } T\in\mathcal{T}.$$

Minimising over $\mathcal{T}$ and interchanging the two minimisations, both of which run over finite sets, gives

$$\min_{T\in\mathcal{T}} \mathrm{wc}(T) \;=\; \min_{\pi\in\Pi}\min_{T\in\mathcal{T}}\left[\Gamma\pi + \sum_{e\in E(T)} c_e^{\pi}\right] \;=\; \min_{\pi\in\Pi}\left[\Gamma\pi + \min_{T\in\mathcal{T}} \sum_{e\in E(T)} c_e^{\pi}\right],$$

the second step because $\Gamma\pi$ does not depend on the tree. At fixed π the vector c^{π} is one cost vector for all trees alike, so the inner problem is the deterministic minimum spanning tree problem (2.1) under c^{π}, of value $\mathrm{MST}(c^{\pi})$. This proves the identity.

For the second claim, let π^* attain the outer minimum and let T^* be a minimum spanning tree under c^{π^*}. Then

$$\min_{T\in\mathcal{T}} \mathrm{wc}(T) \;=\; \Gamma\pi^* + \sum_{e\in E(T^*)} c_e^{\pi^*} \;\geq\; \min_{\pi\in\Pi}\left[\Gamma\pi + \sum_{e\in E(T^*)} c_e^{\pi}\right] \;=\; \mathrm{wc}(T^*) \;\geq\; \min_{T\in\mathcal{T}} \mathrm{wc}(T),$$

so equality holds throughout and T^* is min-max optimal.

It remains to bound the running time. The set Π holds at most $m+1$ levels, one deviation per edge together with zero. A single level costs $O(m \log n)$ time: assembling c^{π} needs one subtraction per edge, and Kruskal's algorithm then returns $\mathrm{MST}(c^{\pi})$ in $O(m \log n)$ time (Section 2.3). Sweeping the levels and retaining the smallest value therefore takes $O(m^2 \log n)$ time in total. □

Under interval uncertainty a single minimum spanning tree computation settled the problem (Corollary 3.1); under a budget, at most $m+1$ of them do. The adversary's choice of where to spend, which changes from tree to tree, has been reduced to a single level, and levels are few enough to enumerate.

Worked Example. The micro-graph of Figure 2.2 shows both how the enumeration of Theorem 3.6 runs and what the budget buys as it grows. We take nominal costs at the interval midpoints and maximum deviations at the half-widths,

$$\bar{c} = (3, 4, 4, 5, 6), \qquad \hat{c} = (1, 1, 3, 1, 1),$$

so that $\bar{c}$ coincides entry by entry with the midpoint scenario $c^{(3)}$ of Table 2.1, while $\bar{c} + \hat{c} = (4, 5, 7, 6, 7)$ agrees with the upper-bound scenario $c^{(2)}$, the vector u of Section 3.2. Neither coincidence belongs to the budgeted model, which is fixed by $\bar{c}$, $\hat{c}$ and Γ alone; both follow from how we have instantiated it here. They are convenient nonetheless, because the two ends of the budget range will return values that Table 2.1 has already recorded. We first work through one budget completely and then let the budget vary.

Fix the budget $\Gamma = 2$. Lemma 3.2 evaluates each tree directly: the worst-case cost of a tree is its nominal cost plus its two largest deviations, so for the three representative trees

$$\mathrm{wc}(T_1) = 11 + 3 + 1 = 15, \quad \mathrm{wc}(T_2) = 12 + 1 + 1 = 14, \quad \mathrm{wc}(T_3) = 14 + 3 + 1 = 18.$$

The remaining five spanning trees have worst-case costs between 16 and 19, so T_2 is the unique min-max tree, at 14.

Theorem 3.6 reaches the same tree without evaluating any tree individually. Its candidate levels are

$$\Pi \;=\; \{0\} \cup \{\hat{c}_e : e \in E\} \;=\; \{0, 1, 3\},$$

three rather than the six that $m + 1$ permits, since the repeated deviations collapse to one candidate. Table 3.1 performs the three deterministic solves. Its outer rows hold two cost vectors the chapter has met before: at $\pi = 0$ nothing is discounted and $c^0 = u$, while at $\pi = 3$ every deviation is absorbed and $c^3 = \bar{c}$. The middle row wins: the level $\pi^* = 1$ gives the total $12 + 2 \cdot 1 = 14$, with minimum spanning tree T_2. The two routes agree, in the value and in the tree.

Table 3.1: The reformulation of Theorem 3.6 on the micro-graph at $\Gamma = 2$. Each row solves one deterministic minimum spanning tree problem under the modified costs c^π; the smallest total identifies the optimal value and an optimal tree.

π	$c^\pi = \bar{c} + [\hat{c} - \pi]^+$	$\mathrm{MST}(c^\pi)$	$\Gamma\pi$	Total
0	$(4, 5, 7, 6, 7)$	15	0	15
1	$(3, 4, 6, 5, 6)$	12	2	**14**
3	$(3, 4, 4, 5, 6)$	11	6	17

One budget exercises the machinery; the range of budgets shows the model. Figure 3.3 records the optimal value at every integer budget, and we read it from left to right. Table 3.1 keeps working throughout: only its flat-payment column depends on Γ, so the same three rows answer each budget that follows.

At the left end, $\Gamma = 0$, the optimum is 11, attained by T_1: the deterministic minimum spanning tree under $\bar{c}$, whose value appears in Table 2.1 as $\mathrm{MST}(c^{(3)})$. At this budget the flat payments vanish, so the $\mathrm{MST}(c^\pi)$ column alone decides: its smallest entry, 11 at the level $\pi = 3$, is the optimum. The level thus moves opposite to the budget here: with nothing to spend, the cheapest account is the one that absorbs every deviation. The level is not a budget in disguise, and reading it as one inverts its role.

One unit of budget already moves the tree. At $\Gamma = 1$ the optimum is 13, attained by T_2, while T_1 falls to second place: $\mathrm{wc}(T_1) = 11 + 3 = 14$ against $\mathrm{wc}(T_2) = 12 + 1 = 13$. The reason is e_3, an edge of T_1 whose deviation 3 is the largest in the graph. T_1 is therefore the cheapest tree when no deviation can occur, and among the most exposed as soon as one can; the optimum switches to T_2 at $\Gamma = 1$ and stays there.

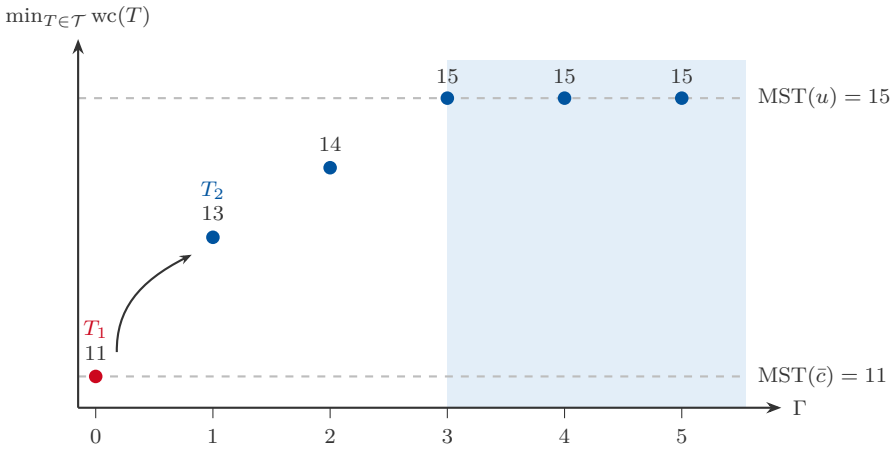

Figure 3.3: Optimal budgeted min-max value on the micro-graph at each integer budget Γ. The dashed lines mark the deterministic optimum under $\bar{c}$ and the interval optimum under u; the shaded region is $\Gamma \geq n - 1$, where every tree's worst case equals its interval worst case. The red point is the only budget at which T_1 is optimal; from $\Gamma = 1$ onwards the optimum is attained by T_2 (arrow: the switch).

At the right end the value saturates. From $\Gamma = 3$ onwards the optimum is 15, which is MST(u) from Section 3.2: every tree's worst case now equals its interval worst case. Saturation arrives at $\Gamma = n - 1 = 3$, well before the $\Gamma = |E| = 5$ at which Definition 2.3 reaches the full box, because a spanning tree of this graph offers only three edges on which to spend. Exactly at the saturation budget the attaining level is no longer unique, $\pi = 0$ and $\pi = 1$ both yielding 15, the tie anticipated after Lemma 3.3; beyond it, nothing changes.

Taken together, the three readings show the budget acting as a dial between the deterministic problem and the interval model. Its ends reproduce two optima already on record, in Table 2.1 and Section 3.2, while in between the value climbs and, more tellingly, the optimal tree changes: a single parameter carries the problem from one model to the other and alters the decision on the way.

Summary

The three uncertainty models of this chapter are separated by a single question: how far the adversary's best answer depends on the tree it faces. Under interval uncertainty it does not depend on the tree at all. Lemma 3.1 identifies one cost vector, the upper bounds, that is simultaneously worst for every tree, so the worst-case cost is a sum of fixed per-edge weights and the outer minimisation is a single deterministic minimum spanning tree computation (Corollary 3.1).

Under discrete uncertainty the dependence is genuine. A tree that is safe in one scenario may be exposed in another, so which scenario witnesses the worst case is a property of the tree, and in general no fixed vector can stand in for the scenario set. The trade-off this forces is what the reductions of Section 3.3 exploit, and two scenarios already suffice to make the problem weakly NP-hard.

Under budgeted uncertainty the answer depends on the tree as well, since the budget is spent on the chosen tree's own largest deviations, and yet the problem

remains polynomial. The reason is that a single scalar removes the dependence: once a level is fixed, every tree is costed edge by edge again, and at most $m + 1$ levels can matter (Theorem 3.6).

The dividing line is therefore not linearity of the worst-case cost, which two of the three models lack in general. It is whether the dependence on the tree can be stripped away by a bounded amount of extra work: none is needed under intervals, one enumerated parameter suffices under a budget, and under discrete scenarios no such device can exist unless $\mathsf{P} = \mathsf{NP}$. The extremal characterisations, the two hardness reductions and the budgeted reformulation are worked through here; the pseudo-polynomial algorithm, the approximation scheme and the logarithmic approximation are surveyed from the literature and attributed where they are stated. Two of the three models return in Chapter 4 under the regret objective, where the ordering changes: the interval case, the straightforward one here, becomes the hard one.

Chapter 4

Min-Max Regret Spanning Tree

4.1 Regret Formulation

4.2 Interval Regret Extremal Behaviour

Lemma 4.1 (Interval Extremal Regret)**.**

Proof. □

4.3 Complexity for Discrete Scenarios

4.3.1 Constant K

Theorem 4.1 (K=2 Weak NP-Hardness)**.**

Proof. □

Theorem 4.2 (Pseudo-Polynomial for Constant K)**.**

4.3.2 Unbounded K

Theorem 4.3 (Strong NP-Hardness)**.**

4.4 Interval Regret Approximation

Theorem 4.4 (Interval NP-Hardness)**.**

Theorem 4.5 (2-Approximation via Midpoint)**.**

Lemma 4.2 (Lower Bound on Regret)**.**

Proof. □

Lemma 4.3 (Upper Bound Relating Solutions)**.**

Proof. □

Proof of Theorem 4.5. □

Summary

Chapter 5

Conclusion

5.1 Synthesis of Results

Table 5.1: Complexity and approximability of robust spanning tree problems. Skeleton; the entries are compiled from Chapters 3 and 4 in the final pass of this chapter.

Uncertainty model	Min-max	Min-max regret
Discrete, constant K		
Discrete, unbounded K		
Interval		
Budgeted		

Complexity Landscape.

Key Patterns.

5.2 Example Gallery

Micro-Graph Solutions Across Objectives.

Extremal Behaviour Visualisation.

5.3 Limitations

Scope of Uncertainty Models.

Scope of Solution Concepts.

Proof Selection.

5.4 Outlook

Extensions in Uncertainty Modelling.

Extensions in Solution Concepts.

Open Problems.

Appendix A

Notation

This appendix provides a reference for the mathematical notation used throughout the thesis. Symbols are grouped by category with the section of first use indicated.

Symbol	Meaning	First Use
Graph Notation		
$G = (V, E)$	Undirected graph with vertex set V, edge set E	§2.1
$n = \|V\|$, $m = \|E\|$	Number of vertices and edges	§2.1
$\delta(X)$	Cut induced by vertex set $X \subseteq V$	§2.1
Trees and Costs		
$\mathcal{T}$	Set of all spanning trees of G	§2.1
T	Spanning tree, $T \in \mathcal{T}$	§2.1
$c\colon E \to \mathbb{R}$	Edge cost function	§2.1
$c(T)$	Cost of tree T, i.e., $\sum_{e \in T} c_e$	§2.1
$\mathrm{MST}(c)$	Minimum spanning tree cost under c	§2.1
T^*	A minimum spanning tree, $c(T^*) = \mathrm{MST}(c)$	§2.1
C_f	Fundamental cycle of non-tree edge f	§2.1
$\delta(X_e)$	Fundamental cut of tree edge e	§2.1
Uncertainty Models		
$\mathcal{U}$	Uncertainty set (discrete, interval, or budgeted)	§2.4
K	Number of discrete scenarios	§2.4
$c^{(k)}$	Scenario k cost vector, $k \in \{1, \ldots, K\}$	§2.4
$[\ell_e, u_e]$	Interval cost bounds for edge e	§2.4
ℓ_e, u_e	Lower and upper bounds on edge cost (interval model)	§2.4
$\mathcal{U}^\Gamma$	Budgeted uncertainty set	§2.4
Γ	Deviation budget (budgeted model)	§2.4
$\bar{c}_e$	Nominal cost of edge e (budgeted model)	§2.4

Continued on next page

Symbol	Meaning	First Use
$\hat{c}_e$	Maximum deviation of edge e (budgeted model)	§2.4
δ_e	Fraction of its maximum deviation taken by edge e	§2.4
$\mathbb{R}^{\lvert E\rvert}_{\geq 0}$	Non-negative cost vectors, required by all three models	§2.4
$u = (u_e)_{e \in E}$	Upper-bound cost vector, to which the interval model reduces	§3.2
π	Level above which deviations are charged (budgeted reformulation)	§3.4
Π	Candidate levels $\{0\} \cup \{\hat{c}_e : e \in E\}$	§3.4
$[z]^+$	Positive part, $\max\{z, 0\}$	§3.4
Robust Objectives		
$\mathrm{Regret}(T, c)$	Regret of T under c: $c(T) - \mathrm{MST}(c)$	§2.4
$\mathrm{wc}(T)$	Worst-case cost of tree T: $\max_{c \in \mathcal{U}} c(T)$	§3.1
Complexity		
P	Polynomial-time complexity class	§2.5
NP	Nondeterministic polynomial-time class	§2.5
NP-hard	At least as hard as NP-complete problems	§2.5
Weakly NP-hard	NP-hard with a pseudo-polynomial algorithm	§2.5
Strongly NP-hard	NP-hard even with poly-bounded numeric values	§2.5
Pseudo-polynomial	Algorithm polynomial in input size and numeric magnitudes	§2.5
α-approximation	Algorithm guaranteeing $\mathrm{ALG} \leq \alpha \cdot \mathrm{OPT}$	§2.5
ε	Tolerance parameter for approximation schemes	§2.5
PTAS	Polynomial-time approximation scheme	§2.5
FPTAS	Fully polynomial-time approximation scheme	§2.5

Bibliography

[1] Panos Kouvelis and Gang Yu. *Robust Discrete Optimization and Its Applications*. Vol. 14. Nonconvex Optimization and Its Applications. Dordrecht: Kluwer Academic Publishers, 1997. ISBN: 978-0-7923-4291-5. DOI: 10.1007/978-1-4757-2620-6.

[2] Bernhard Korte and Jens Vygen. *Combinatorial Optimization: Theory and Algorithms*. 6th ed. Vol. 21. Algorithms and Combinatorics. Berlin, Heidelberg: Springer, 2018. ISBN: 978-3-662-56039-6. DOI: 10.1007/978-3-662-56039-6.

[3] Christina Büsing. 'Combinatorial Optimization – Optimierung B (Skript OptiB)'. Lecture notes, Version: February 4, 2023. RWTH Aachen, 2023.

[4] Ravindra K. Ahuja, Thomas L. Magnanti and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Upper Saddle River, NJ: Prentice Hall, 1993. ISBN: 978-0-13-617549-0.

[5] Marc Goerigk and Michael Hartisch. *An Introduction to Robust Combinatorial Optimization: Concepts, Models and Algorithms for Decision Making under Uncertainty*. Vol. 361. International Series in Operations Research & Management Science. Cham: Springer, 2024. DOI: 10.1007/978-3-031-61261-9.

[6] Hassene Aissi, Cristina Bazgan and Daniel Vanderpooten. 'Min–max and min–max regret versions of combinatorial optimization problems: A survey'. In: *European Journal of Operational Research* 197.2 (2009), pp. 427–438. DOI: 10.1016/j.ejor.2008.09.012.

[7] Francisco Barahona and William R. Pulleyblank. 'Exact arborescences, matchings and cycles'. In: *Discrete Applied Mathematics* 16.2 (1987), pp. 91–99. DOI: 10.1016/0166-218X(87)90067-9.

[8] Hassene Aissi, Cristina Bazgan and Daniel Vanderpooten. 'Approximation complexity of min-max (regret) versions of shortest path, spanning tree, and knapsack'. In: *Algorithms – ESA 2005: 13th Annual European Symposium*. Vol. 3669. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2005, pp. 862–873. DOI: 10.1007/11561071_76.

[9] Adam Kasperski and Paweł Zieliński. 'On the approximability of minmax (regret) network optimization problems'. In: *Information Processing Letters* 109.5 (2009), pp. 262–266. DOI: 10.1016/j.ipl.2008.11.001.

[10] Adam Kasperski and Paweł Zieliński. 'On the approximability of robust spanning tree problems'. In: *Theoretical Computer Science* 412.4–5 (2011), pp. 365–374. DOI: 10.1016/j.tcs.2010.10.006.

[11] Werner Baak, Marc Goerigk, Adam Kasperski and Paweł Zieliński. 'Robust min-max (regret) optimization using ordered weighted averaging'. In: *European Journal of Operational Research* 322.1 (2025), pp. 171–181. DOI: 10.1016/j.ejor.2024.10.028.

[12] Vittorio Bilò, Ioannis Caragiannis, Angelo Fanelli, Michele Flammini and Gianpiero Monaco. 'Simple Greedy Algorithms for Fundamental Multidimensional Graph Problems'. In: *44th International Colloquium on Automata, Languages, and Programming (ICALP 2017)*. Vol. 80. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2017, 125:1–125:13. DOI: 10.4230/LIPIcs.ICALP.2017.125.

[13] Dimitris Bertsimas and Melvyn Sim. 'Robust discrete optimization and network flows'. In: *Mathematical Programming* 98.1–3 (2003), pp. 49–71. DOI: 10.1007/s10107-003-0396-4.

Declaration on the Use of AI Tools

During the writing of this thesis I used Claude (Anthropic) in order to improve the readability and language of the manuscript. After using this tool/service, I reviewed and edited the content as needed and take full responsibility for the content of this thesis.